

Chapter 3

Transport

~~Layer~~

A note on the

We're making these

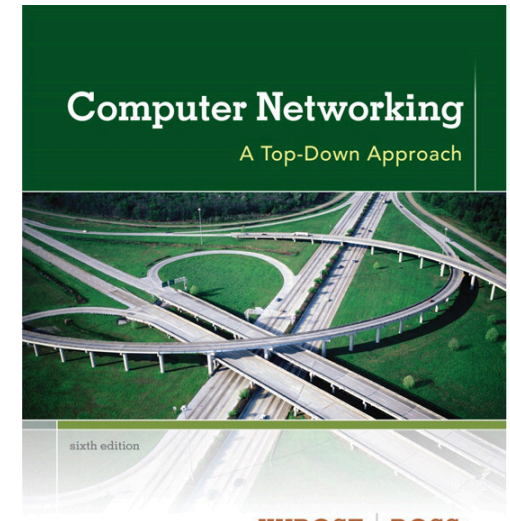
They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs.

They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

©All material copyright 1996-2013
J.F Kurose and K.W. Ross, All Rights Reserved



*orking: A
Top Down
Approach
6th edition
Jim Kurose, Keith
Ross
Addison-Wesley
March 2012*

Chapter 3: Transport Layer

our goals:

- ❖ understand principles behind
 - ❖ learn about Internet transport layer
-
- multiplexing and demultiplexing
 - reliable data transfer
 - flow control
 - congestion control
 - TCP: connection-oriented reliable transport
 - TCP congestion control

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

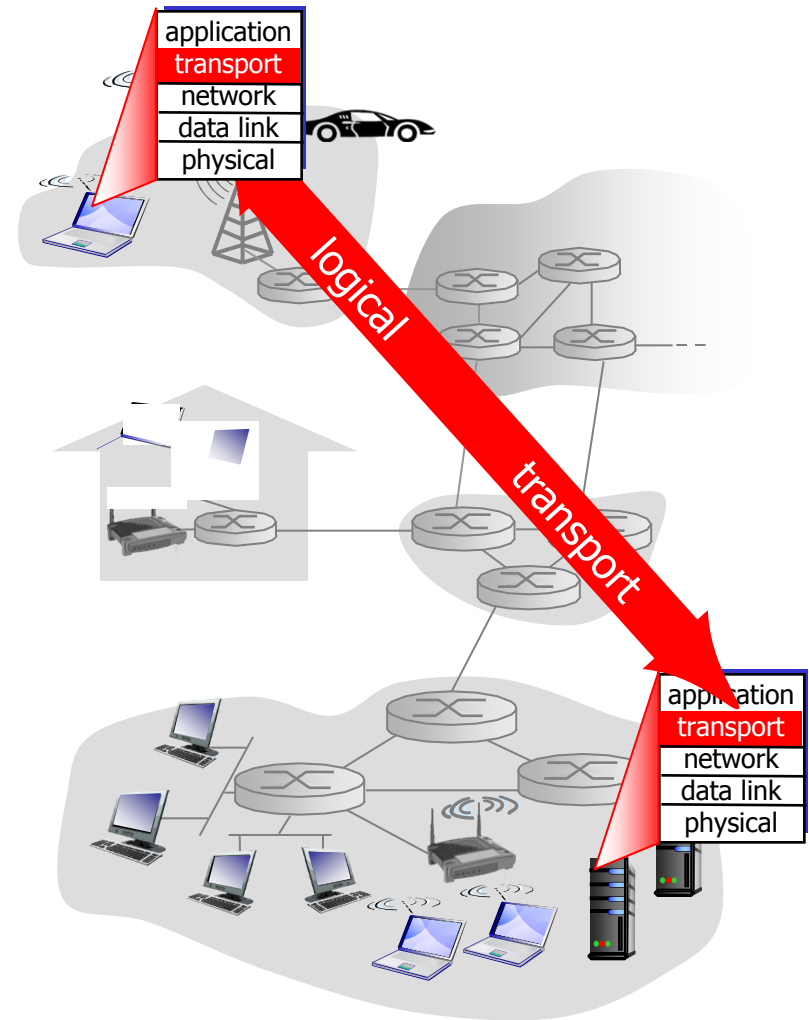
- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

Transport services and protocols

- ❖ provide *logical communication* between app processes running
- ❖ in end
 - send side: breaks app messages into *segments*, passes to network layer
 - rcv side: reassembles segments into messages, passes to app layer



Transport vs. network layer

❖ *network layer:*
logical
communication

logic
communication
between
processes

- relies on,
enhances,
network layer
services

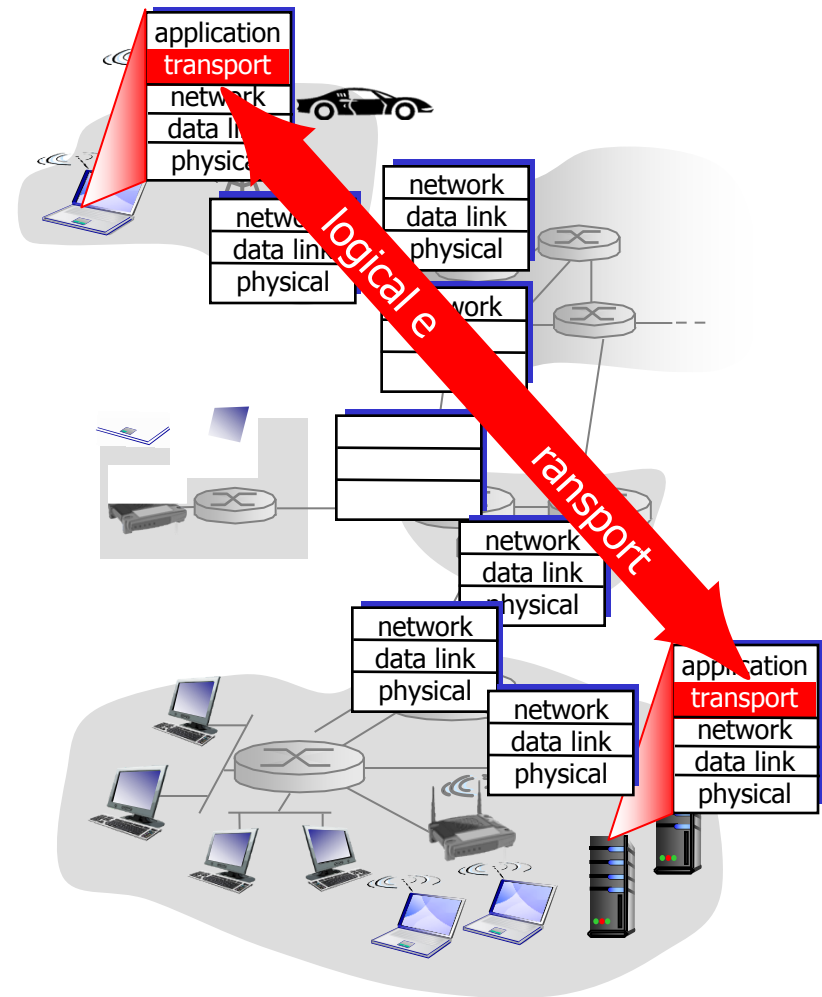
household analogy:

*12 kids in Ann's house
delivering letters to 12*

- ids
- ❖ app messages =
letters in envelopes
- ❖ transport protocol =
Ann and Bill who
demux to in-house
siblings
- ❖ network-layer
protocol = postal
service

Internet transport-layer protocols

- ❖ reliable, in-order delivery (TCP)
 - congestion control
 - flow control
- ❖ unreliable, unordered delivery: UDP
 - no-frills extension of “best-effort” IP
- ❖ services not available:
 - delay guarantees
 - bandwidth



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

Multiplexing/demultiplexing

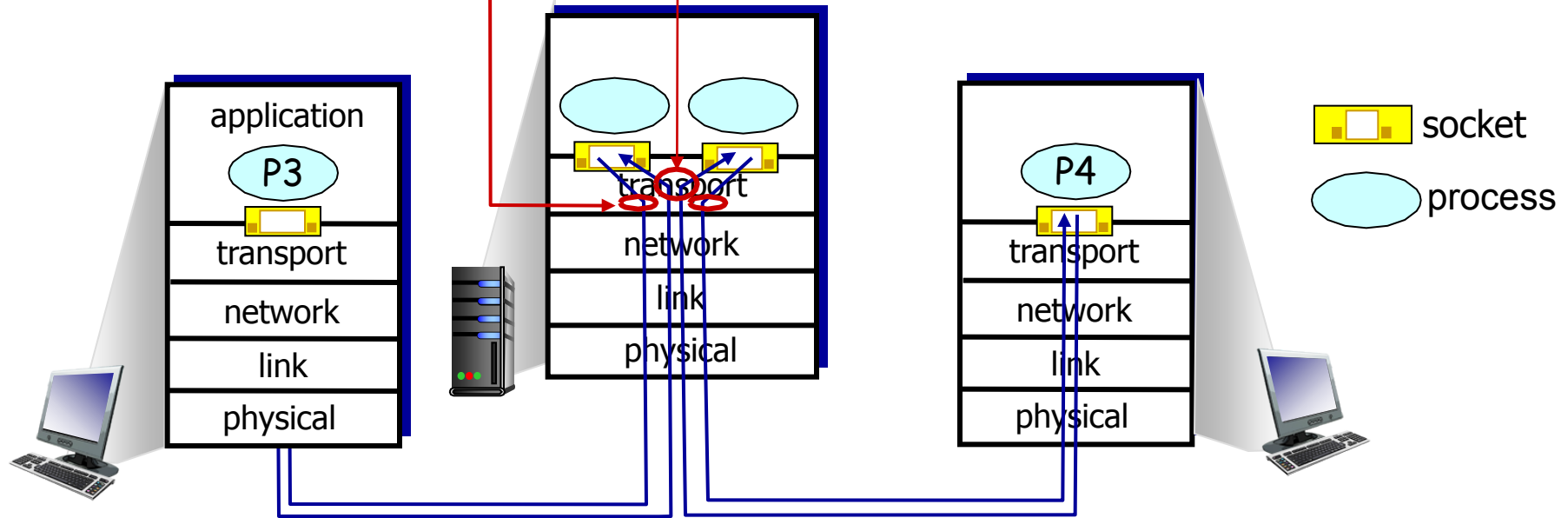
g

multiplexing at sender:

handle data from multiple sockets, add transport header (later used)

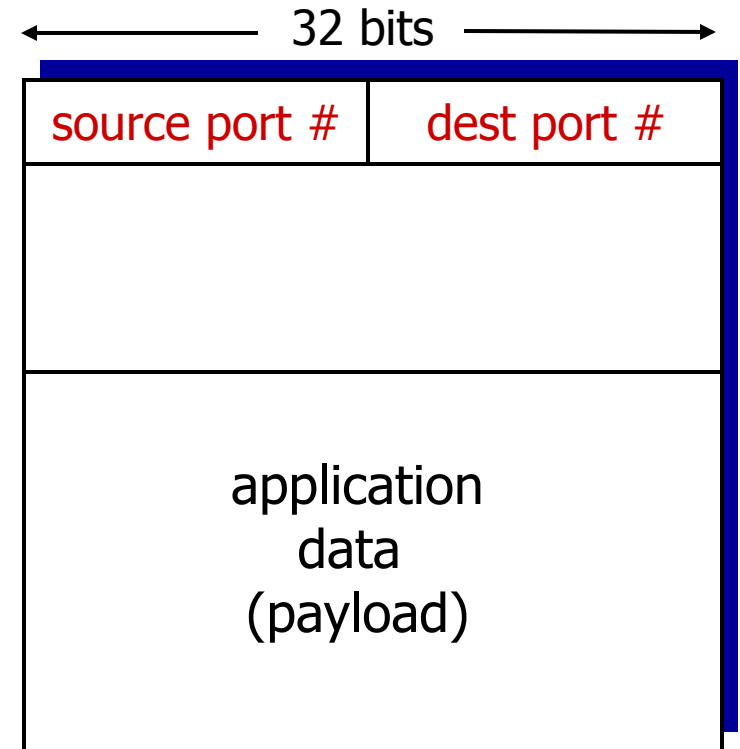
demultiplexing at receiver:

use header info to deliver received segments to correct



How demultiplexing works

- ❖ host receives IP datagrams
 - each datagram
 - each one transport-layer segment
 - each segment has source, destination port number
- ❖ host uses *IP addresses & port numbers* to direct segment to



TCP/UDP segment format

Connectionless demultiplexing

- ❖ *recall*: created socket has host-local port #:

```
DatagramSocket mySocket1  
= new DatagramSock
```

- ❖ *recall*: when creating datagram to send into UDP socket,

ess

-
- ❖ when host receives UDP segment:

- checks destination port # in segment
- directs UDP segment to socket with that port #



IP datagrams with *same dest. port #*, but different source IP addresses and/or source port numbers will be directed to *same socket* at dest

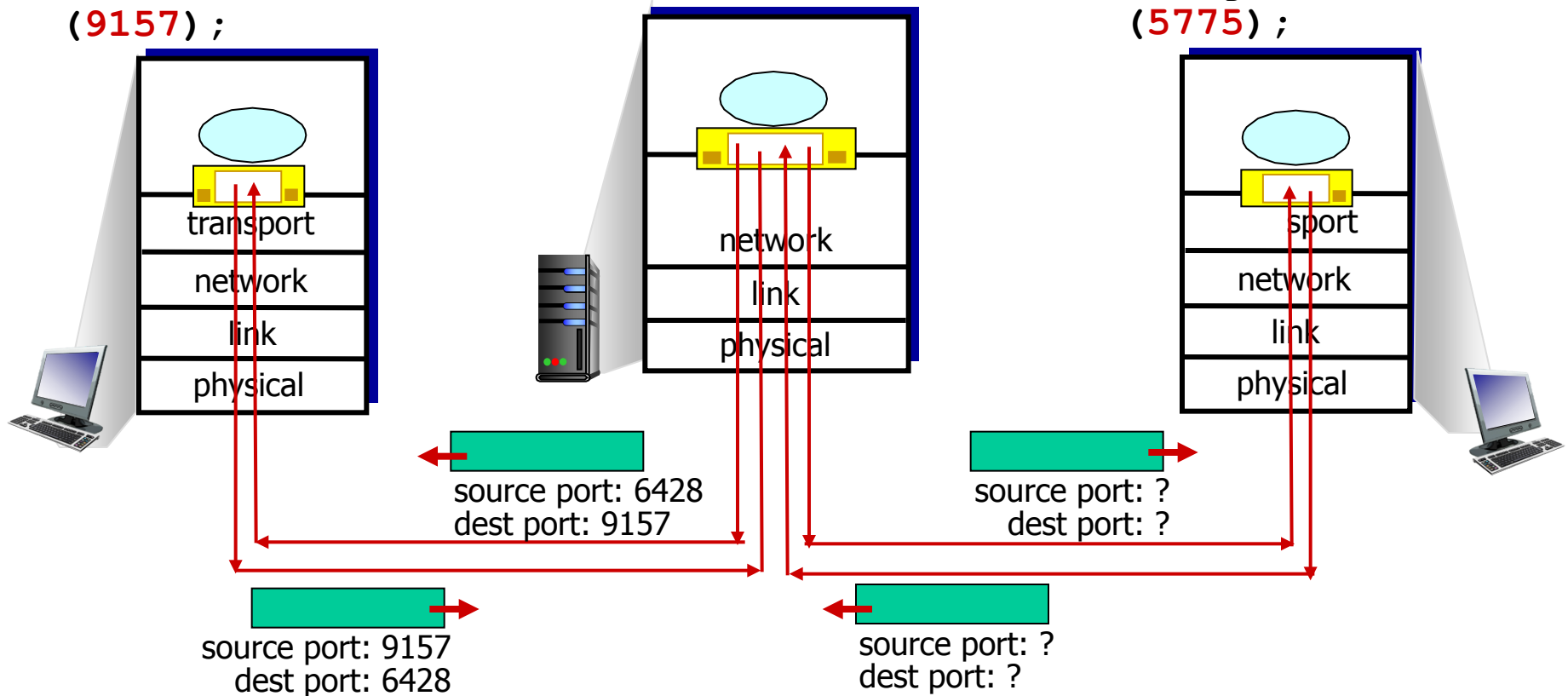
port #

Connectionless demux: example

```
DatagramSocket  
mySocket2 = new  
DatagramSocket  
(9157);
```

```
DatagramSocket  
serverSocket = new  
DatagramSocket  
(6428);
```

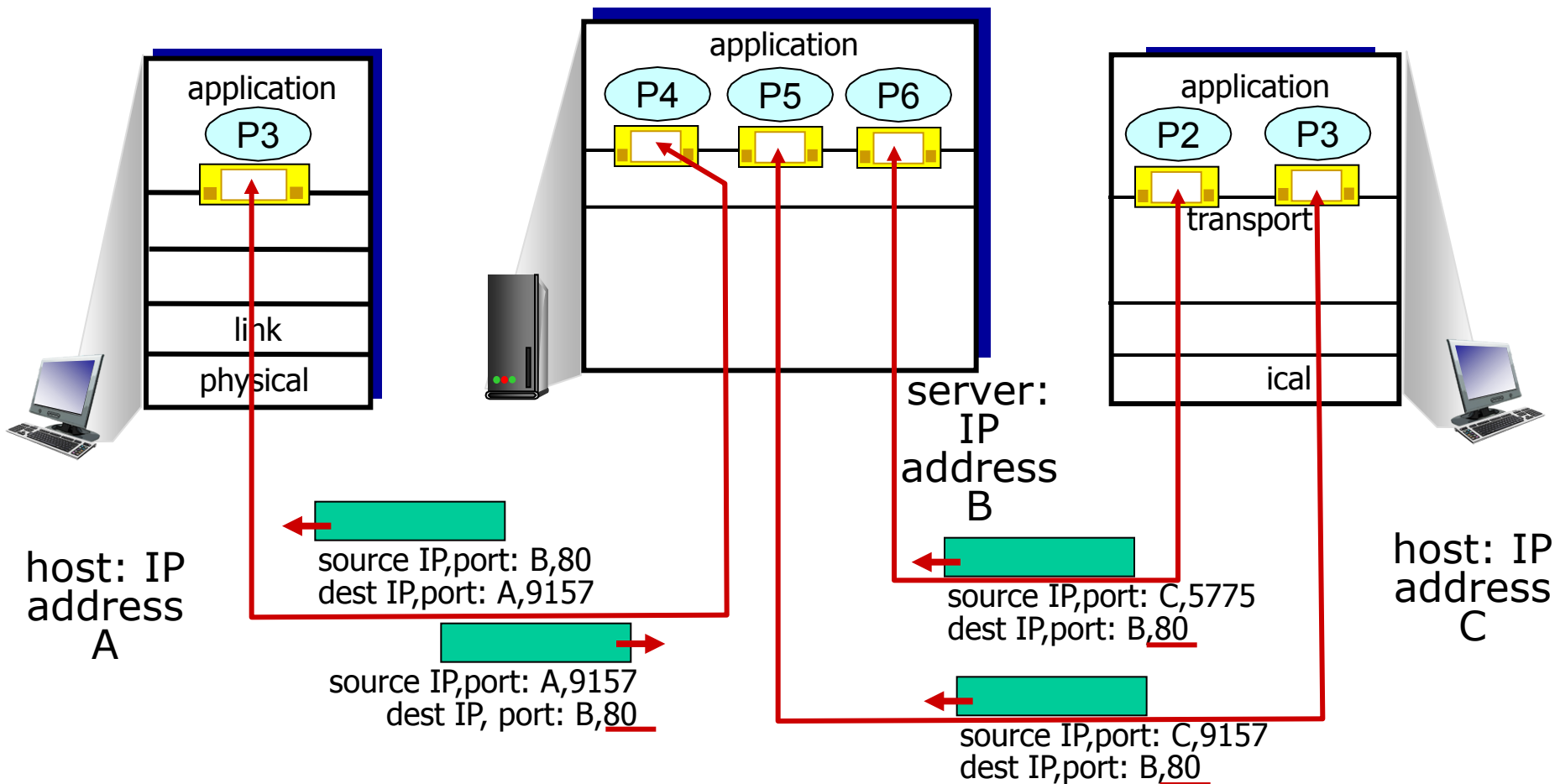
```
DatagramSocket  
mySocket1 = new  
DatagramSocket  
(5775);
```



Connection-oriented demux

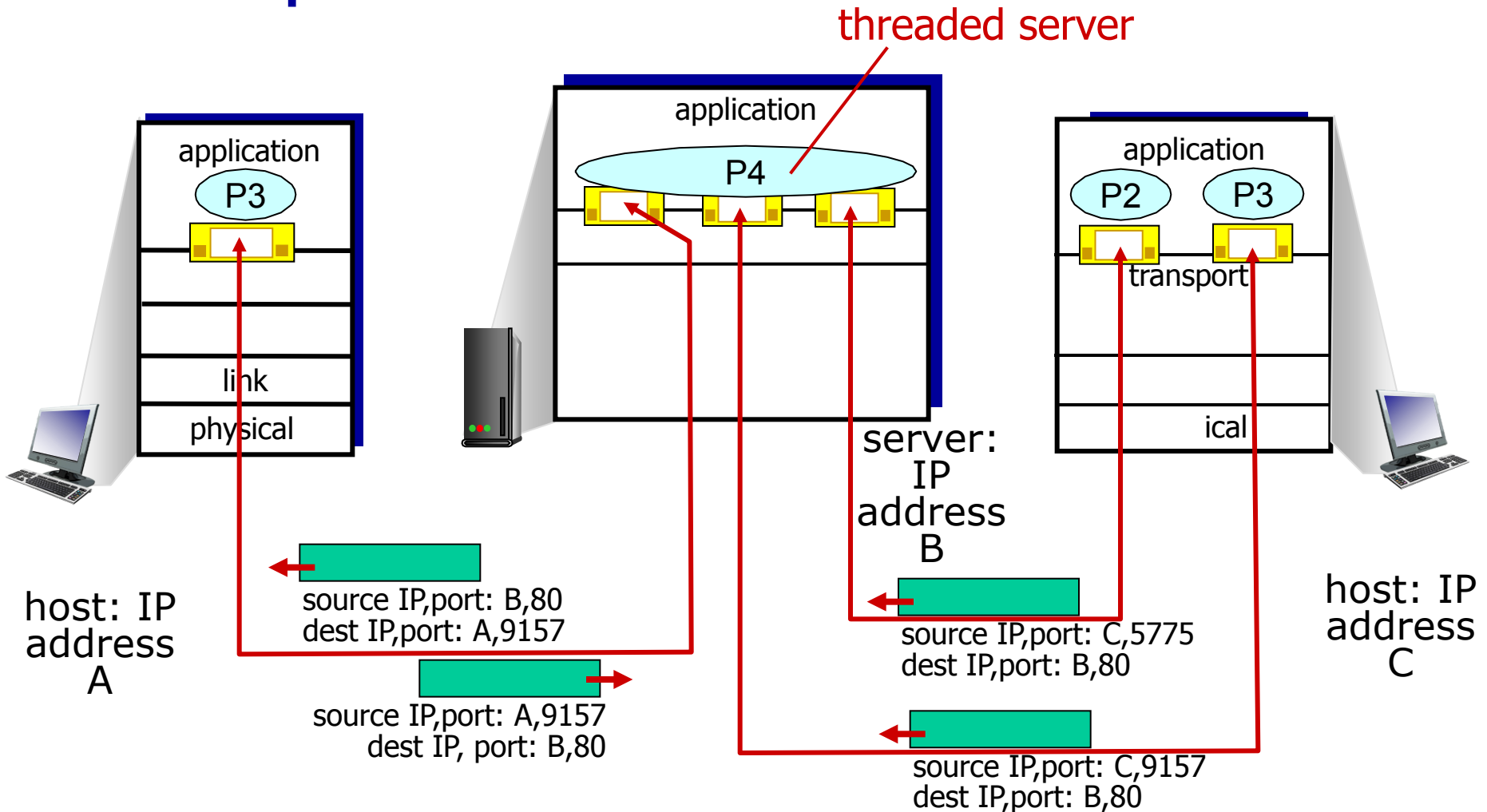
- ❖ TCP socket identified by 4-tuple:
 - dest
 - dest port number
- ❖ demux: receiver uses all four values to direct segment to appropriate socket
- ❖ server host may support many simultaneous TCP connections, each with its own 4-tuple
- ❖ web servers have different sockets for each connecting client
 - non-persistent HTTP will have different socket for each request

Connection-oriented demux: example



three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Connection-oriented demux: example



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport:

- flow control
- connection management

3.6 principles of congestion control

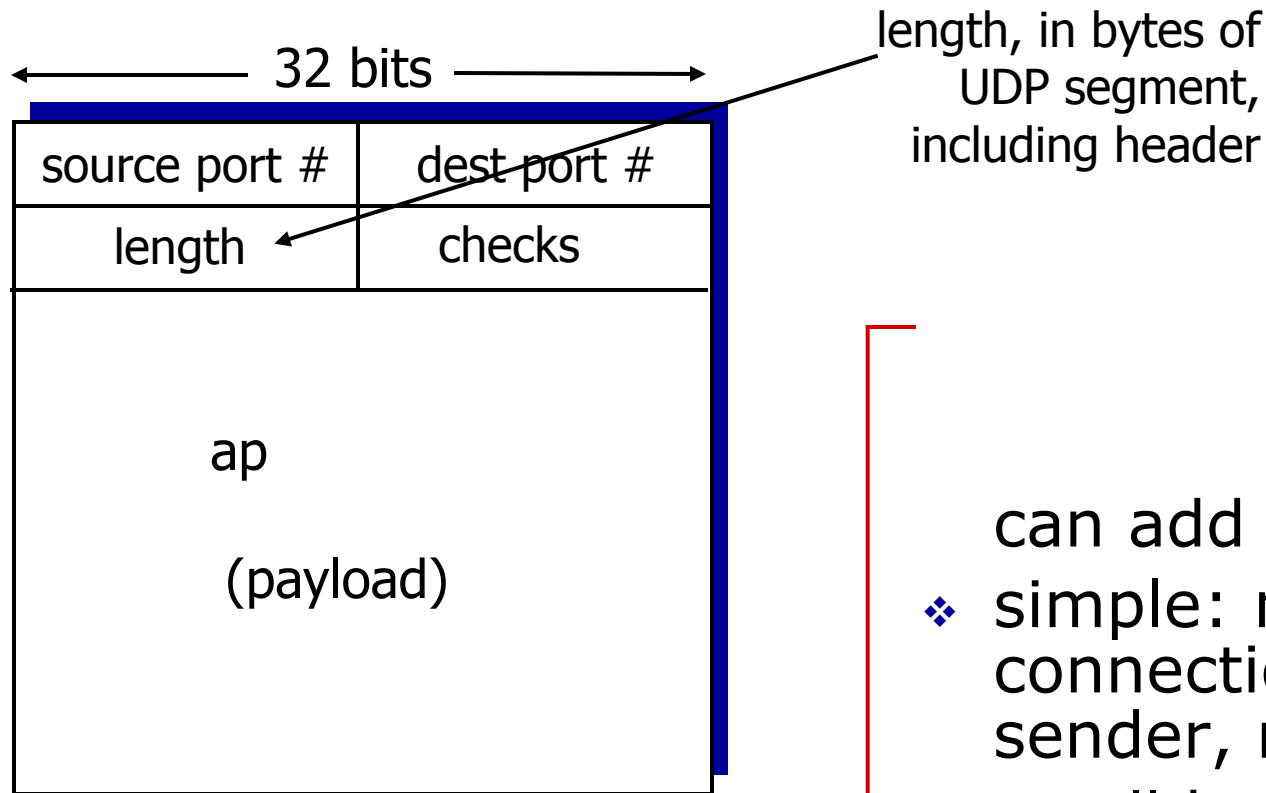
3.7 TCP congestion control

UDP: User Datagram Protocol

[RFC 768]

- ❖ “no frills,” “bare bones” Internet transport protocol
- ❖ “best effort” service
 - lost
 - delivered out-of-order to app
- ❖ *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others
- ❖ UDP use:
 - streaming multimedia apps
 - loss tolerant, rate sensitive
- ❖ reliable transfer over UDP:
 - add reliability at application layer
 - application-specific error recovery!

UDP: segment header



UDP segment format

- ❖ simple: no connection state at sender, receiver
 - ❖ small header size
 - ❖ no congestion control: UDP can blast away as fast as desired
- t (which can add delay)

UDP checksum

Goal: detect “errors” (e.g., flipped bits) in transmitted segment

sender:

- head
sequence of 16-bit integers
- ❖ checksum: addition (one’s complement sum) of segment contents
- ❖ sender puts checksum value into UDP checksum field

er:

- ❖ check if computed checksum equals checksum field value:
 - NO - error detected
 - YES - no error detected. *But maybe errors nonetheless?* More later

Internet checksum: example

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0								0	1	0	1	0	1
wr																
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

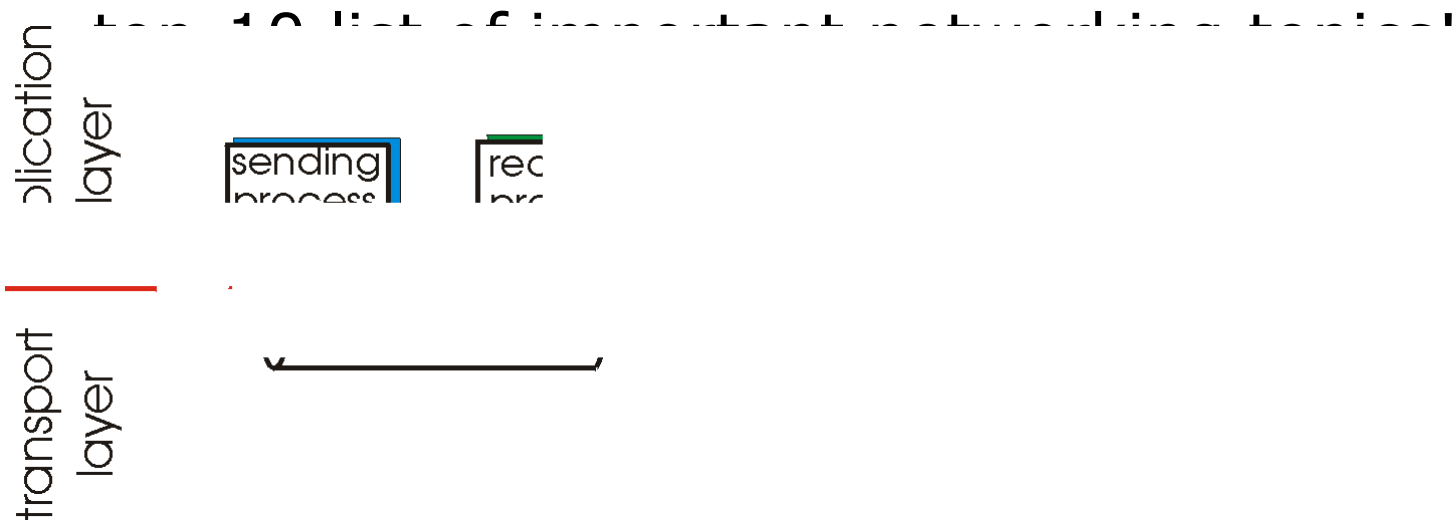
- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

Principles of reliable data transfer

- ❖ important in application, transport, link layers



(a) provided service

- ❖ characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Principles of reliable data transfer

- ❖ important in application, transport, link layers

■ top-10 list of important networking topics!

application
layer

sending
process

receiving
process

transport
layer



(a) provided service

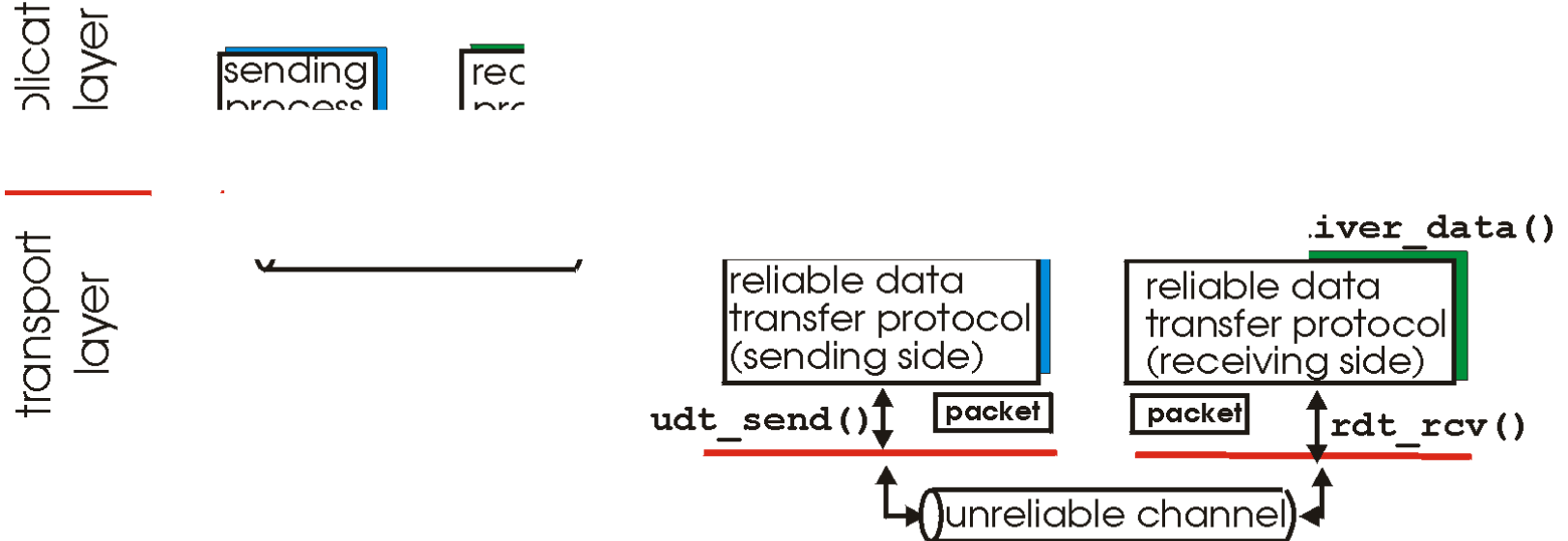
(b) service implementation

- ❖ characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

Principles of reliable data transfer

- ❖ important in application, transport, link layers

■ top-10 list of important networking topics!



(a) provided service

(b) service implementation

- ❖ characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

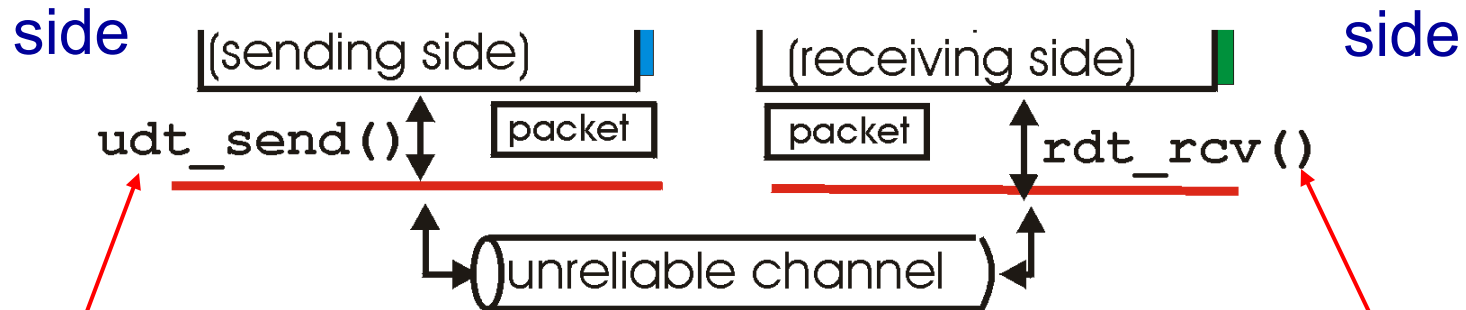
Reliable data transfer: getting started

rdt_send() : called from above, (e.g., by app.). Passed data to deliver to receiver upper layer

rdt_send()

deliver_data() : called by **rdt** to deliver data to upper

deliver_data()



udt_send() : called by rdt, to transfer packet over unreliable channel to receiver

rdt_rcv() : called when packet arrives on rcv-side of channel

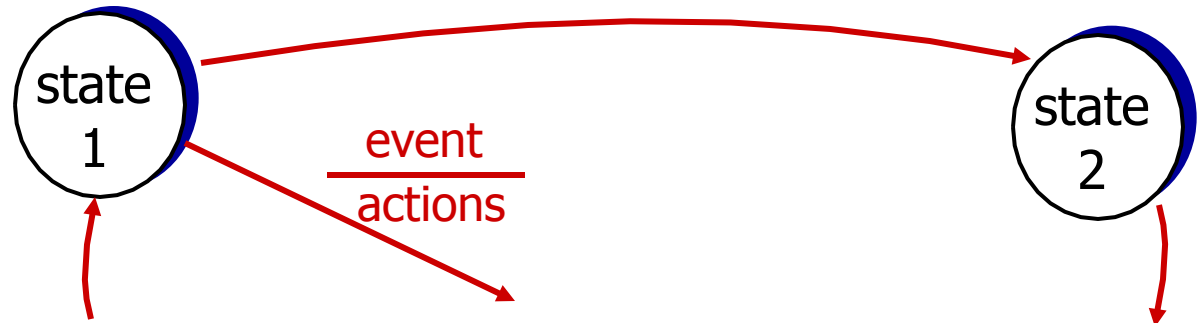
Reliable data transfer: getting started

we'll:

- ❖ incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)

- ❖ use fi
specify sender, receiver

state: when in this "state" next state uniquely determined by next event



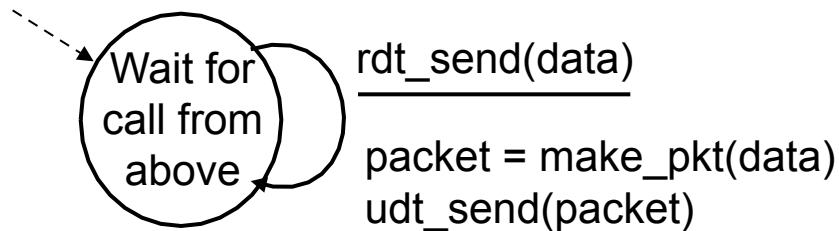
rdt1.0: reliable transfer over a ~~reliable channel~~

- ❖ underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- ❖ separate FSM

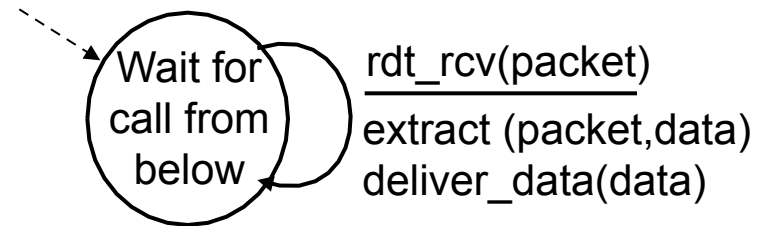
receiver:

▪ rece

nnel



sender



receiver

rdt2.0: channel with bit errors

- ❖ underlying channel may flip bits in packet
 - checksum to detect bit errors

How do humans recover from "errors" during conversation?

rdt2.0: channel with bit errors

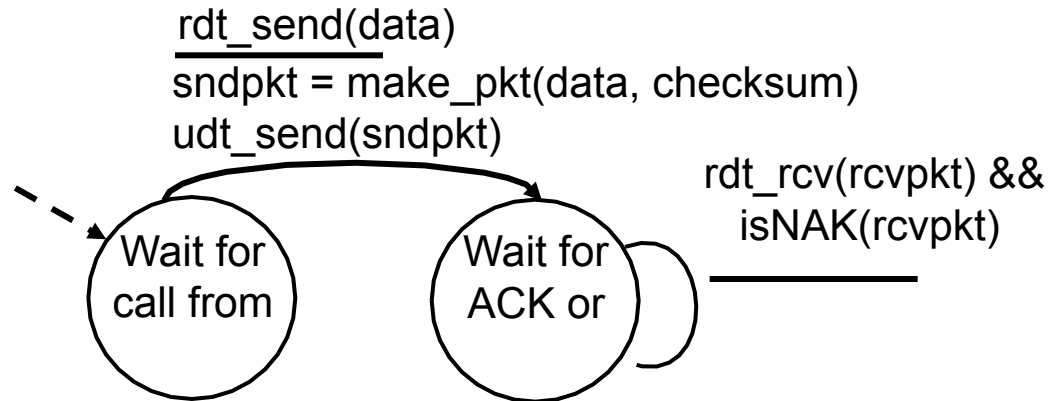
- ❖ underlying channel may flip bits in packet
 - checksum to detect bit errors
- ❖ *the question* er from

ex

OK

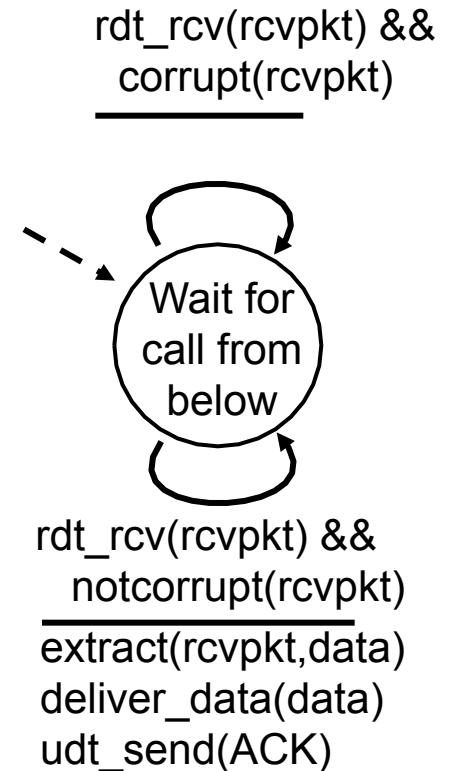
- *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
- sender retransmits pkt on receipt of NAK
- ❖ new mechanisms in `rdt2.0` (beyond `rdt1.0`):
 - error detection
 - feedback: control msgs (ACK,NAK) from receiver to sender

rdt2.0: FSM specification

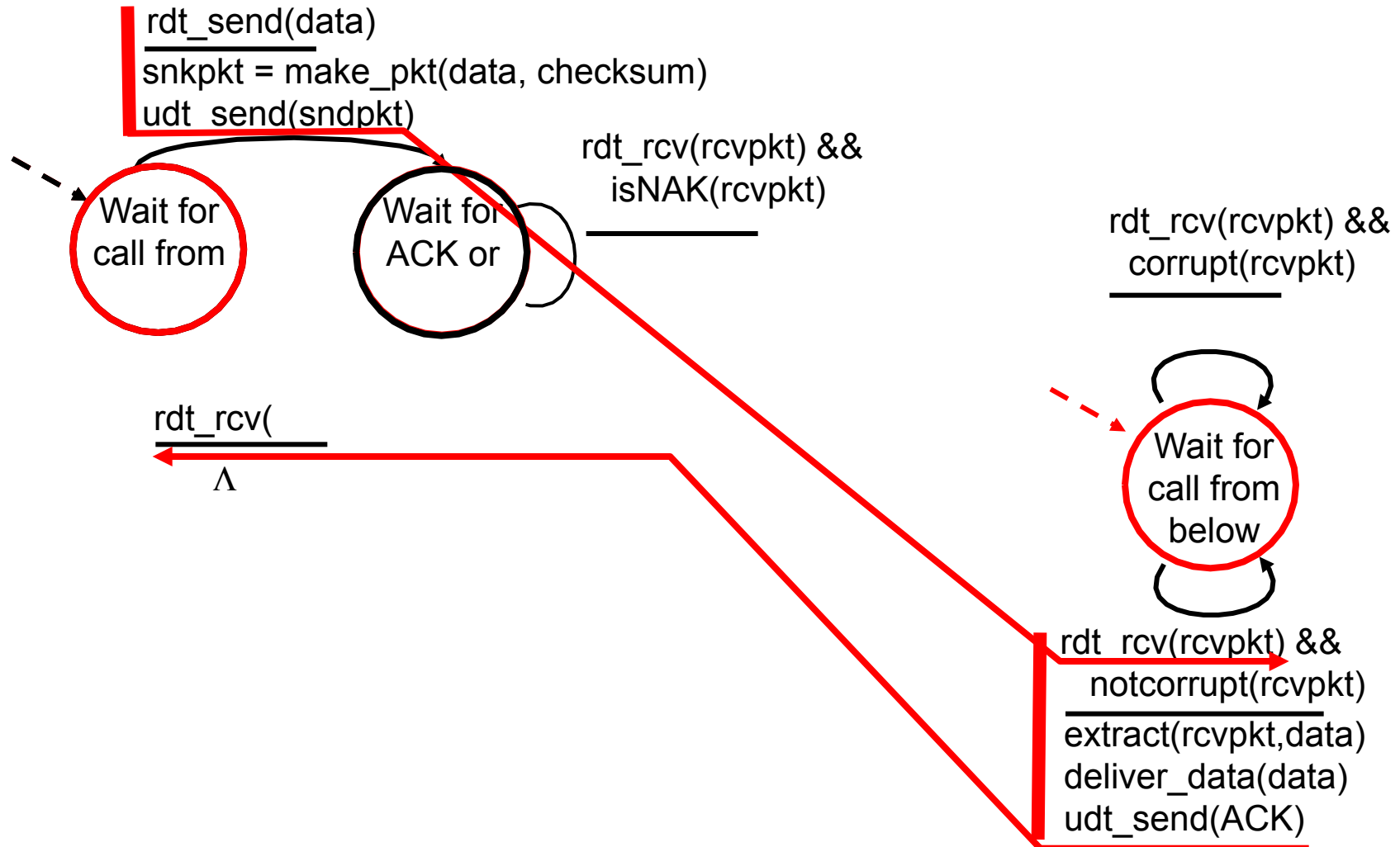


rdt_rcv(
 Λ)
sender

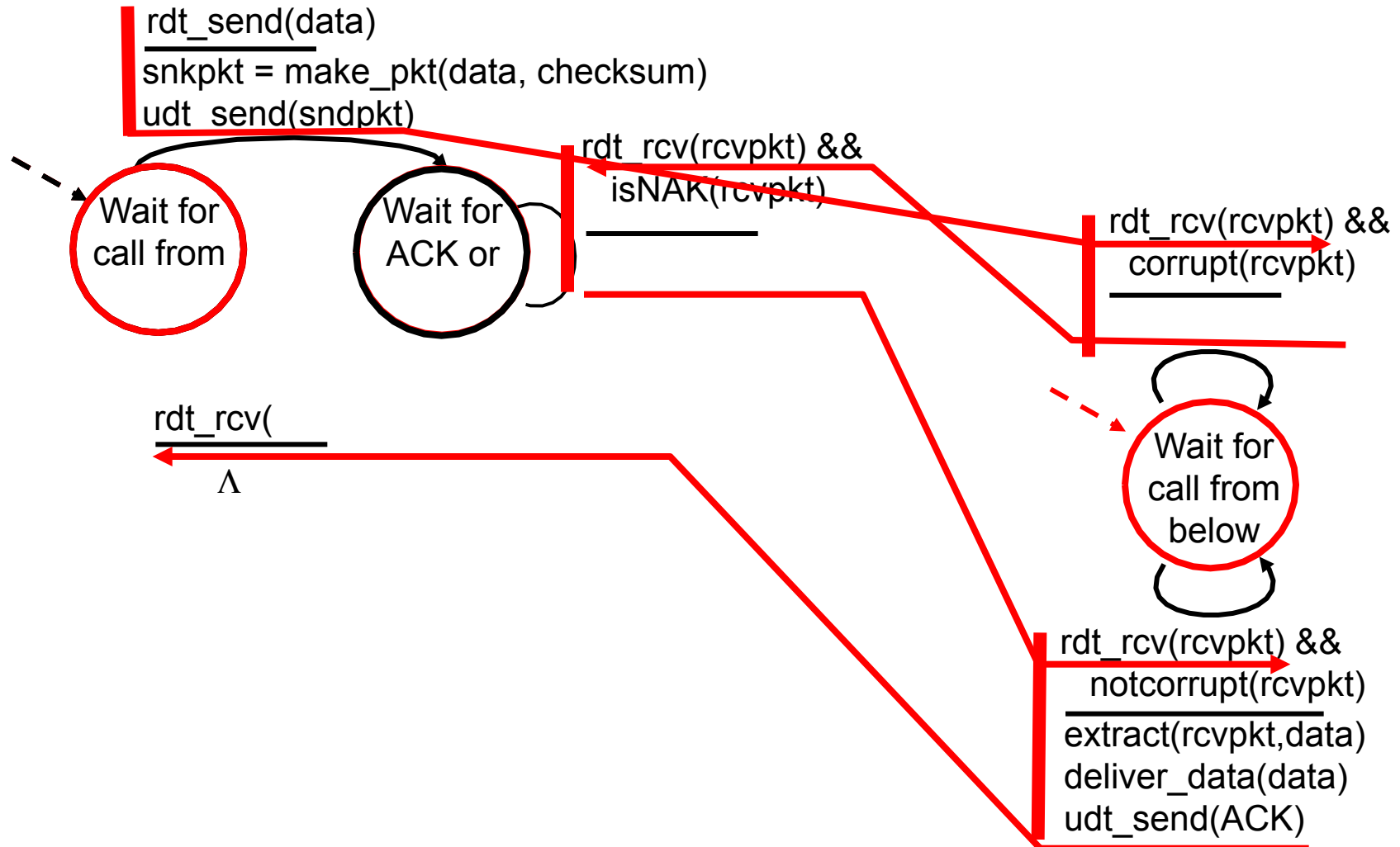
receiver



rdt2.0: operation with no errors



rdt2.0: error scenario



rdt2.0 has a fatal flaw!

what happens if
ACK/NAK
corrupted?

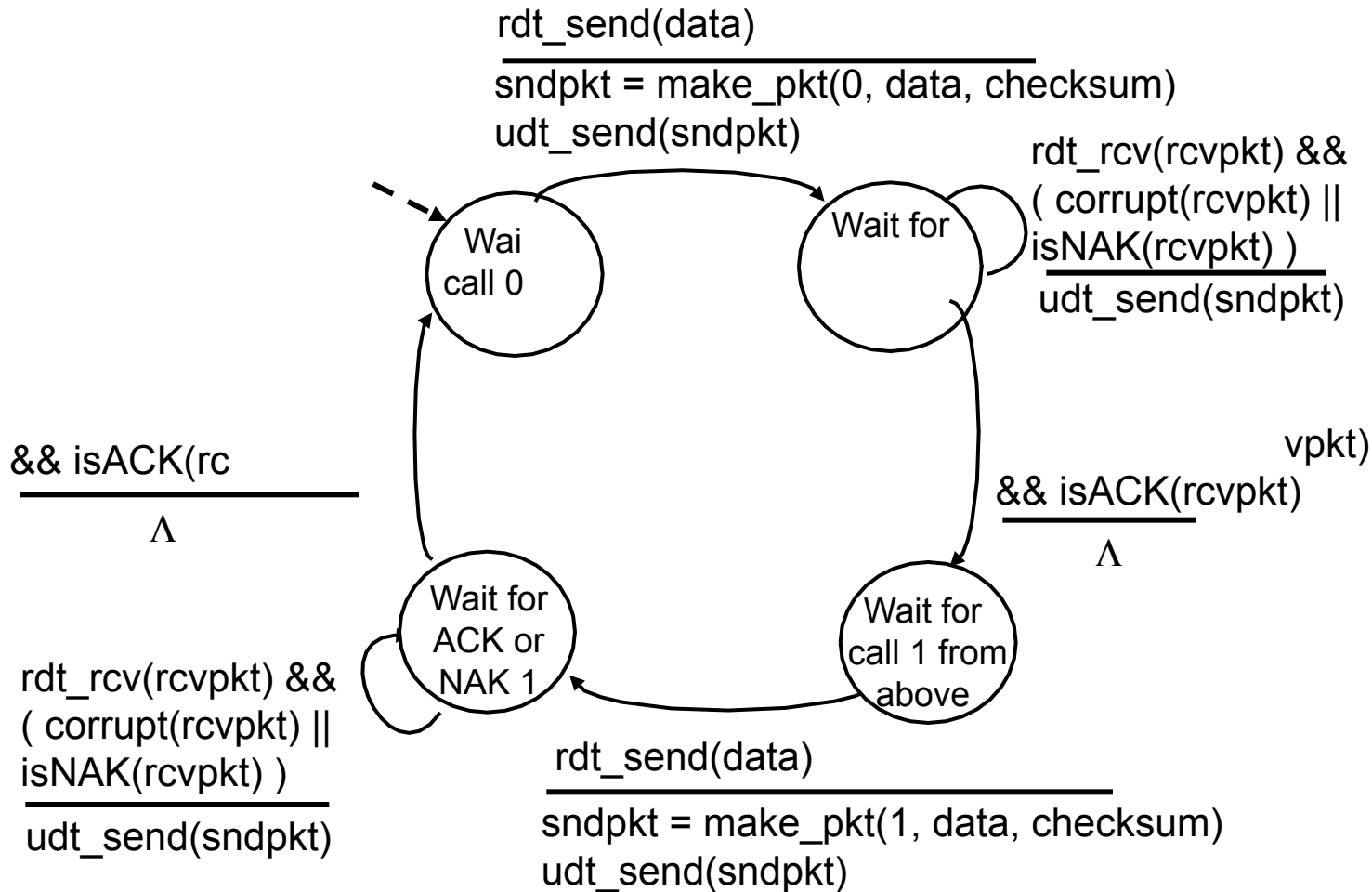
handling
duplicates:
r retransmits

❖ can't j
possible duplicate

sequence number to
each pkt

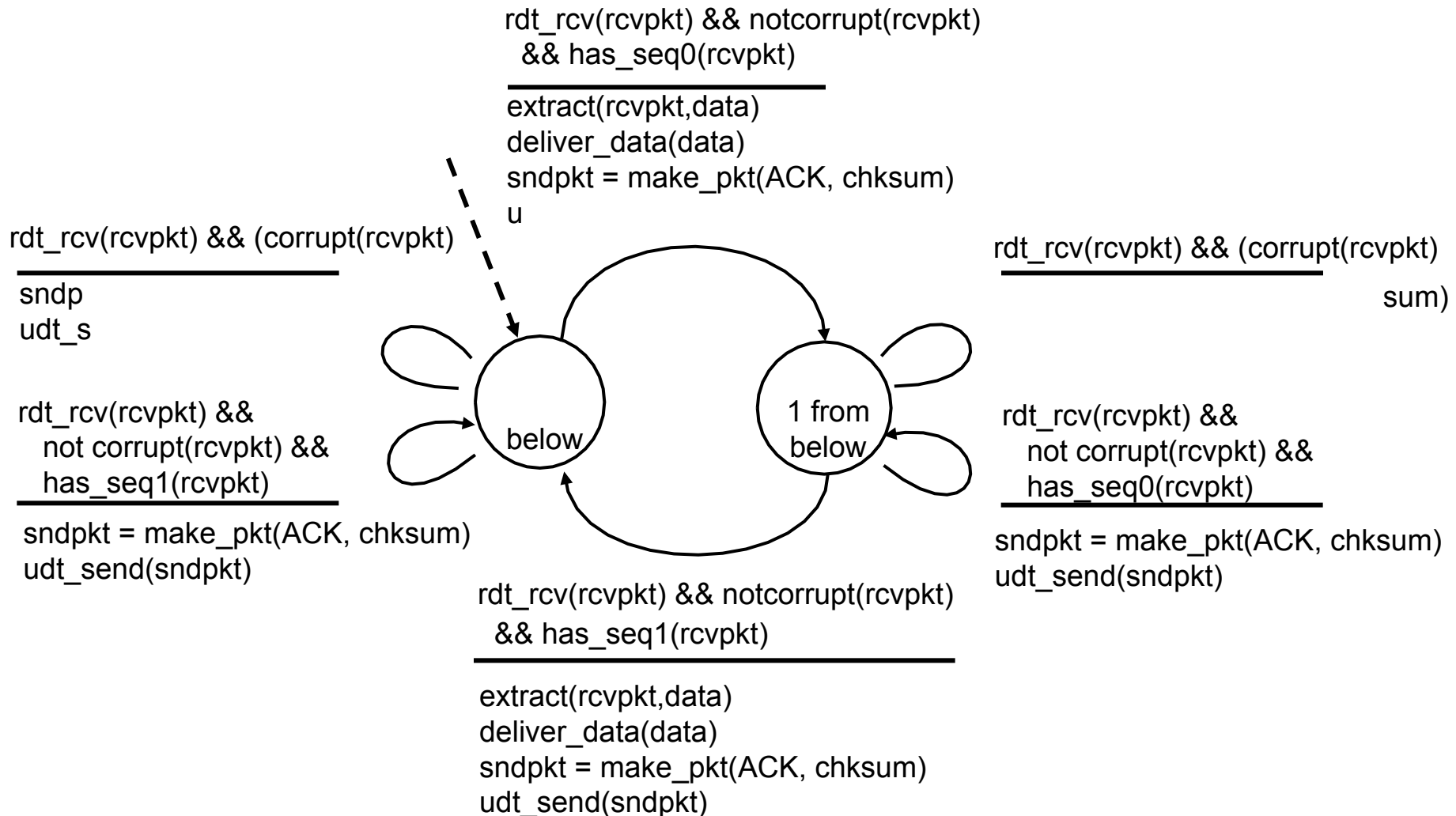
stop and wait receiver discards
sender sends (doesn't deliver up)
packet, duplicate pkt
then waits for
receiver
response

rdt2.1: sender, handles garbled ACK/NAKs



rdt2.1: receiver, handles garbled

~~ACK/NAKs~~



rdt2.1: discussion

sender:

- ❖ seq # added to pkt
- ❖ must check if received ACK/NAK corrupted
- ❖ twice as many states
 - state must “remember” whether “expected” pkt

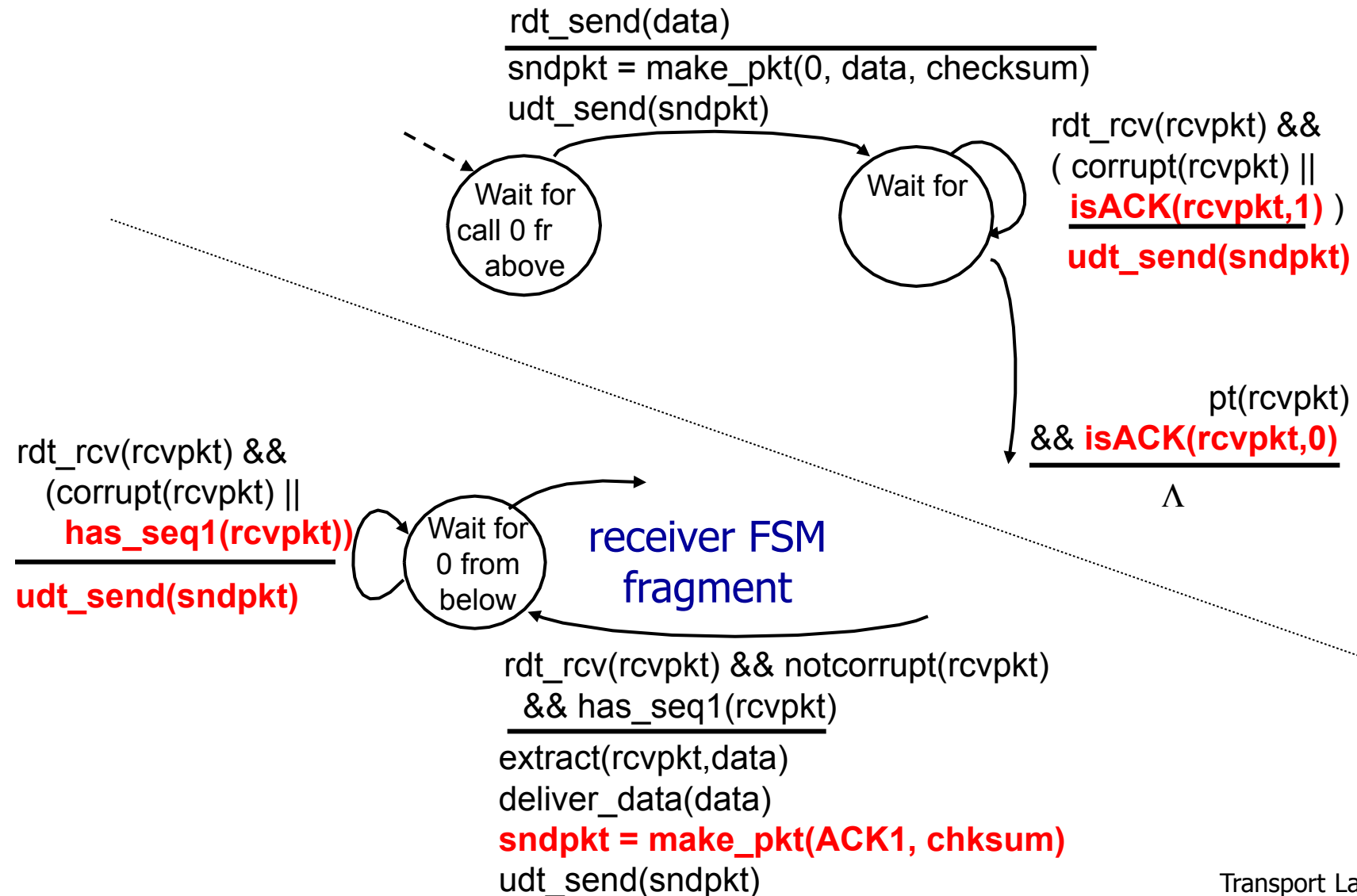
receiver:

- ❖ must check if received packet is
1 is expected pkt seq #
- ❖ note: receiver can *not* know if its last ACK/NAK received OK at sender

rdt2.2: a NAK-free protocol

- ❖ same functionality as rdt2.1, using ACKs only
- ❖ instead of NAK ds ACK for
bein
- ❖ duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

rdt2.2: sender, receiver fragments



rdt3.0: channels with errors *and* loss

new assumption:
underlying
channel can

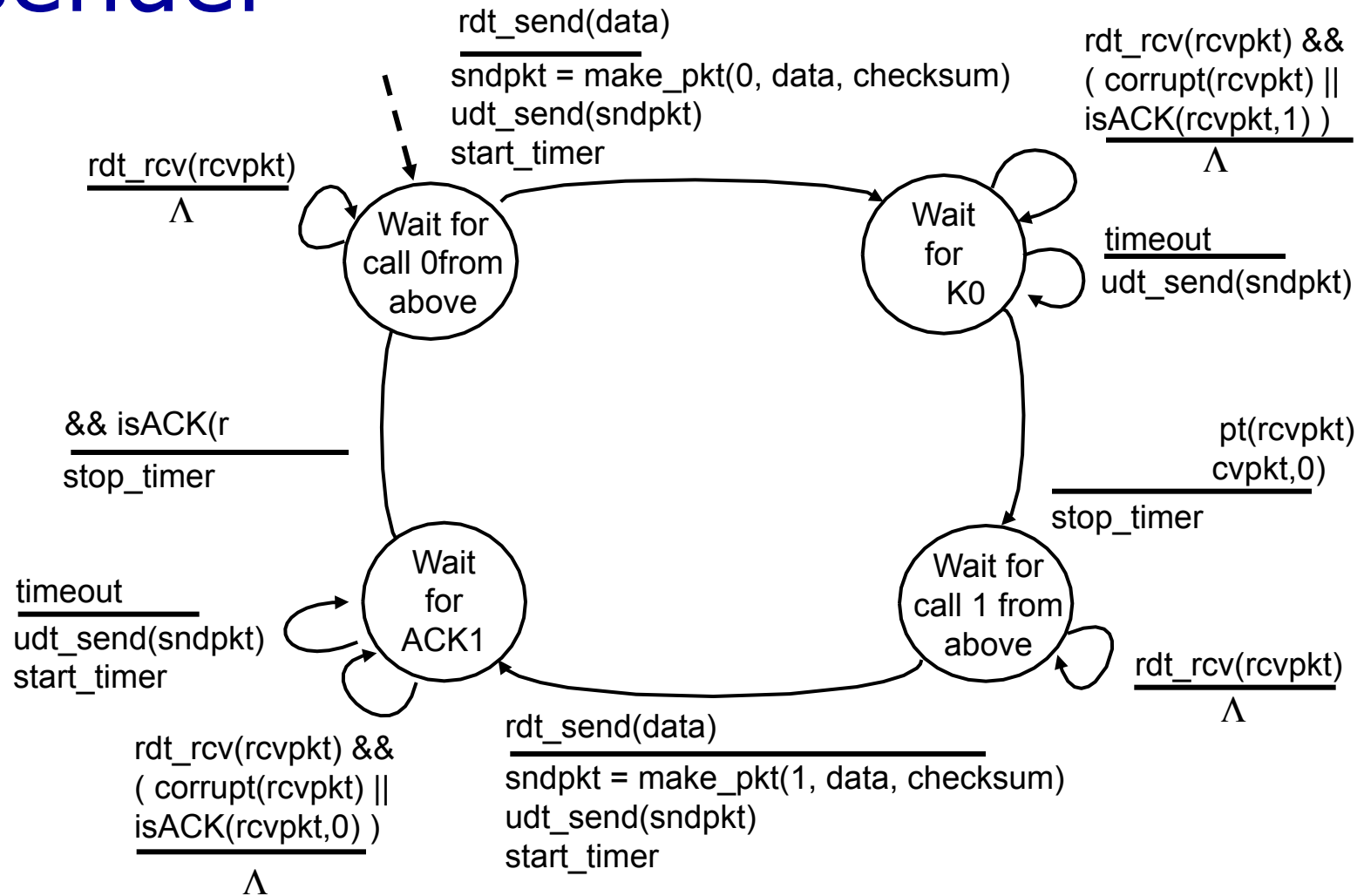
- the
ACKs,
retransmissions
will be of help ...
but not enough

approach: sender
waits “reasonable”
time of time for

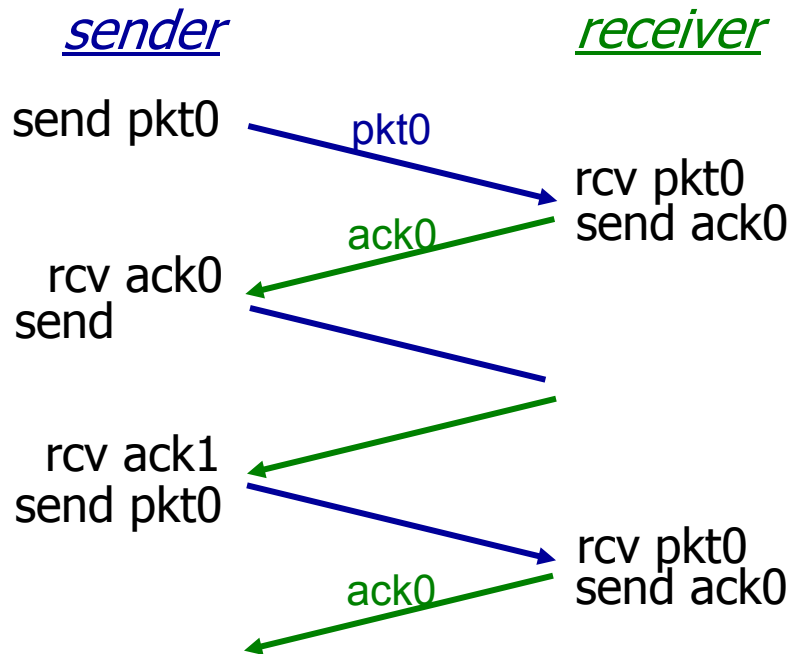
- ❖ if pkt (or ACK) just
delayed (not lost):
 - retransmission will
be duplicate, but
seq. #'s already
handles this
 - receiver must
specify seq # of pkt
being ACKed
- ❖ requires countdown

rdt3.0

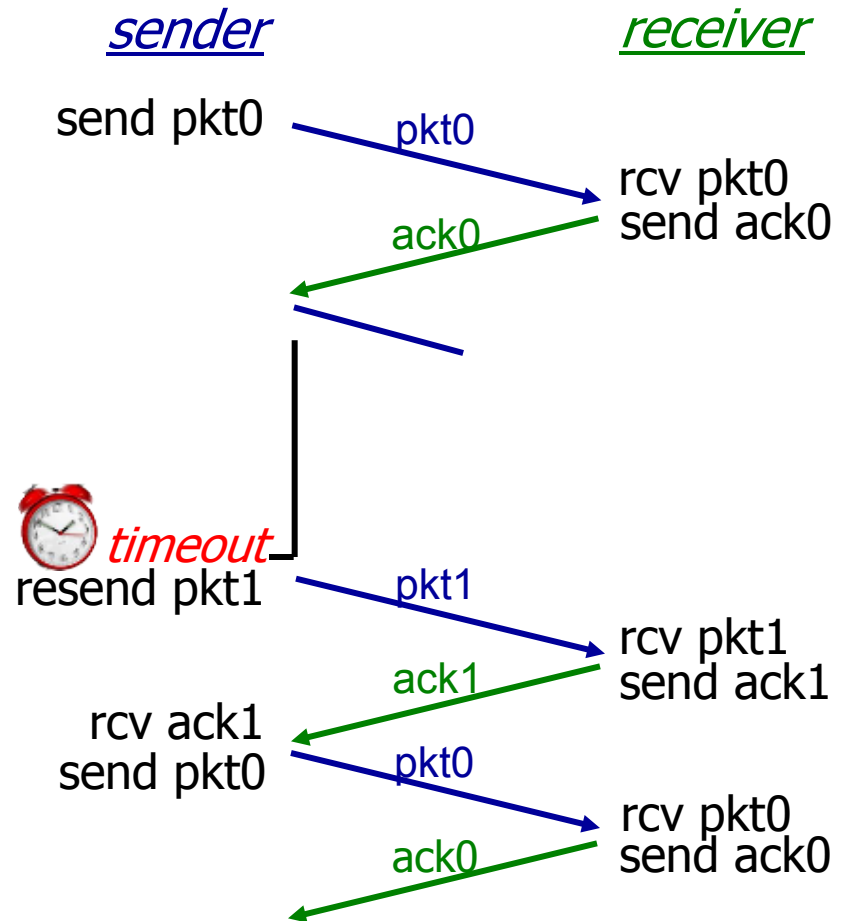
sender



rdt3.0 in ~~action~~



(a) no loss

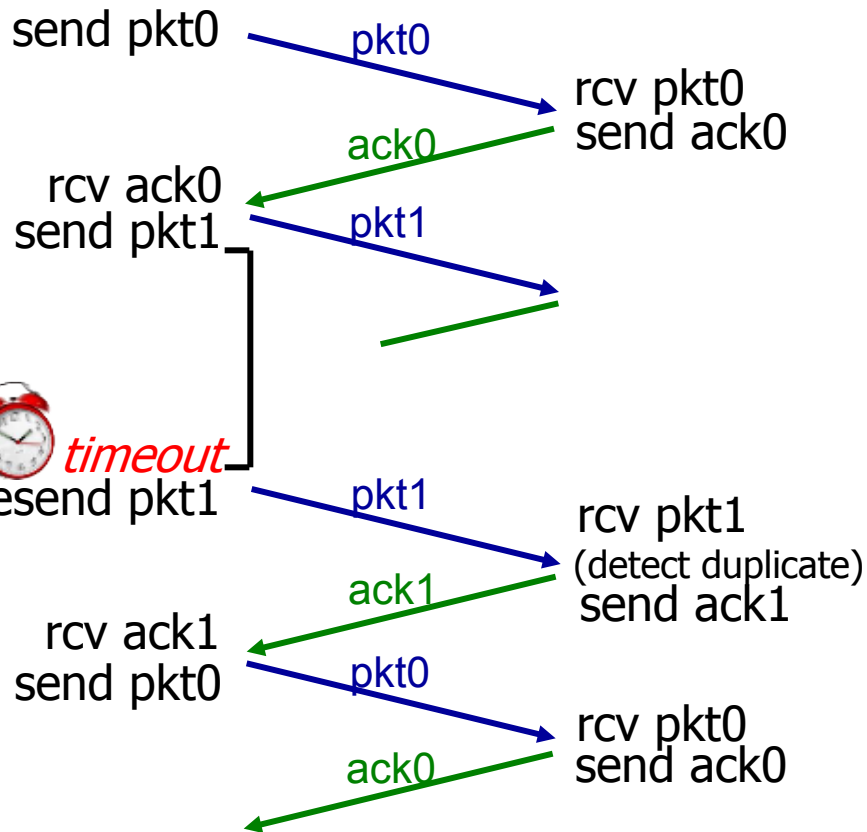


(b) packet loss

rdt3.0 in ~~action~~

sender

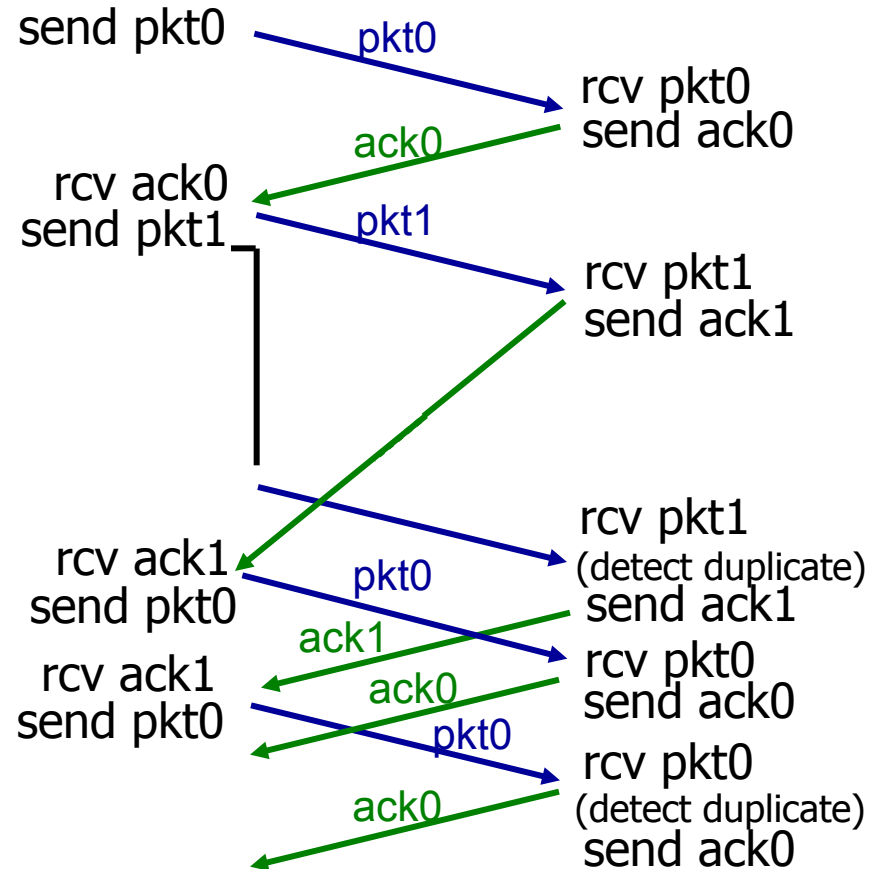
receiver



(c) ACK loss

sender

receiver



(d) premature timeout/ delayed ACK

Performance of rdt3.0

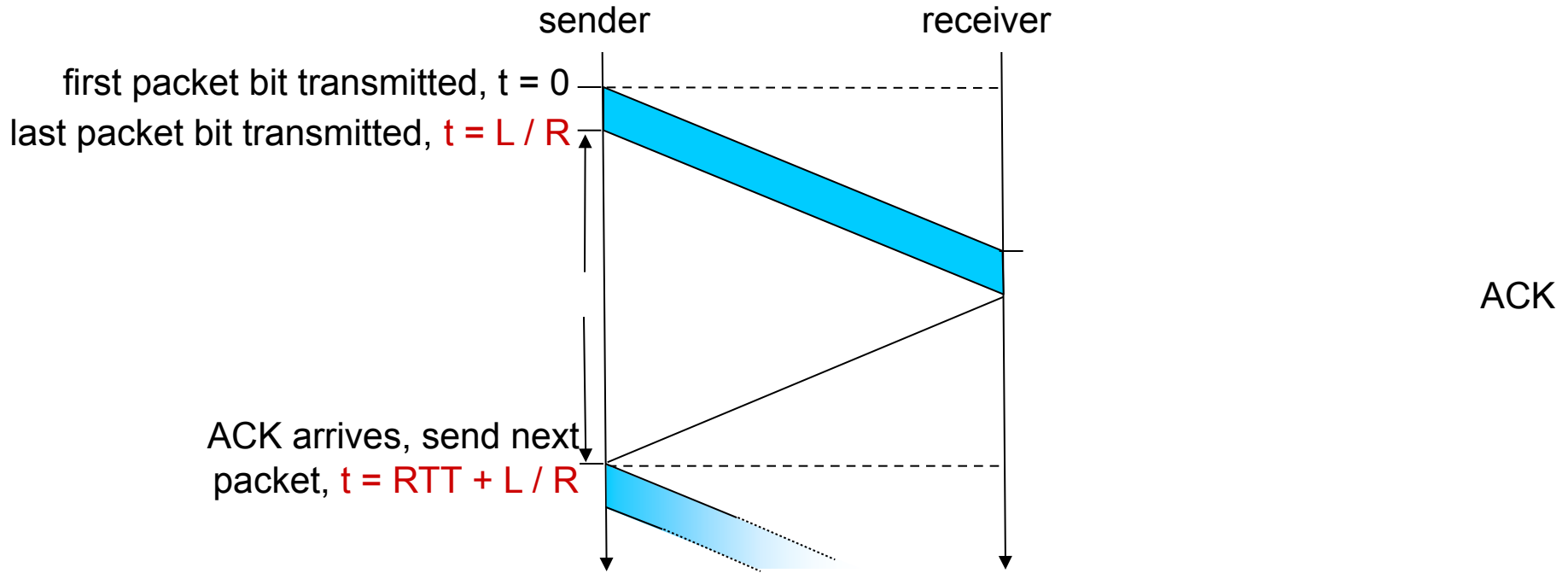
- ❖ rdt3.0 is correct, but performance stinks
- ❖ e.g.: 1 Gbps link, 15 ms prop. delay, 8000 bit packet: L

■ $U_{\text{sender sending}}$ der busy

$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

- if RTT=30 msec, 1KB pkt every 30 msec:
33kB/sec thrupt over 1 Gbps link
- ❖ network protocol limits use of physical resources!

rdt3.0: stop-and-wait operation

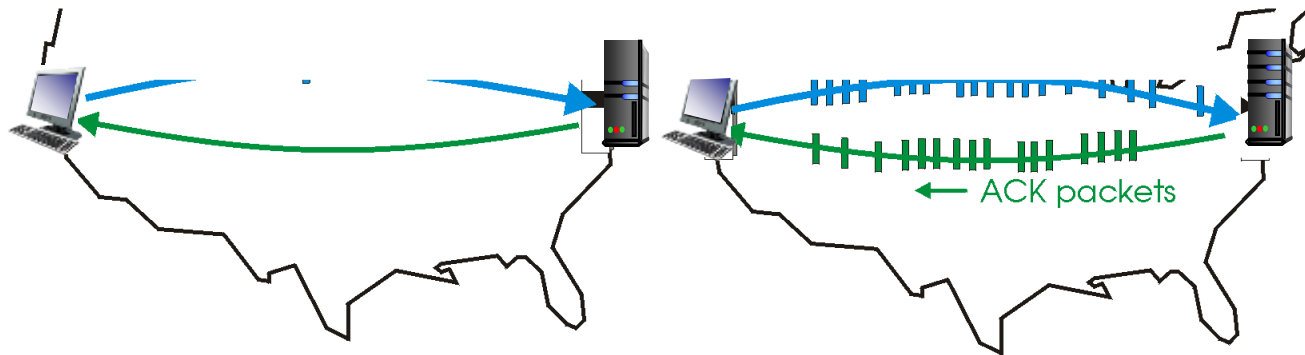


$$U_{\text{sender}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

Pipelined protocols

pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts

- range of sequence numbers must be increased

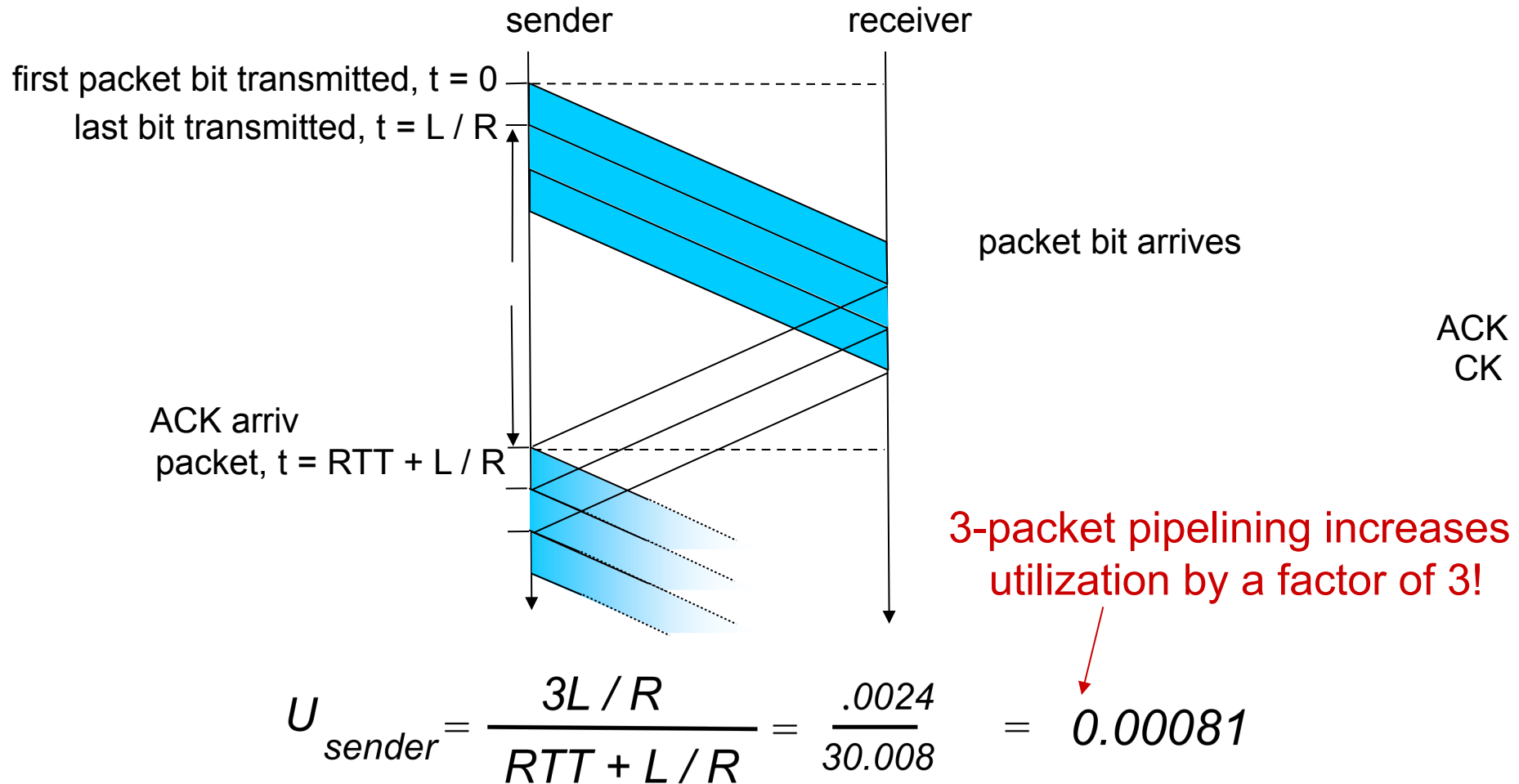


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- ❖ two generic forms of pipelined protocols:
go-Back-N, selective repeat

Pipelining: increased utilization



Pipelined protocols:

overview

Go-back-N:

- ❖ sender can have up to N unacked packets in pi

ack

- doesn't ack packet if there's a gap
- ❖ sender has timer for oldest unacked packet
 - when timer expires, retransmit *all* unacked packets

Selective Repeat:

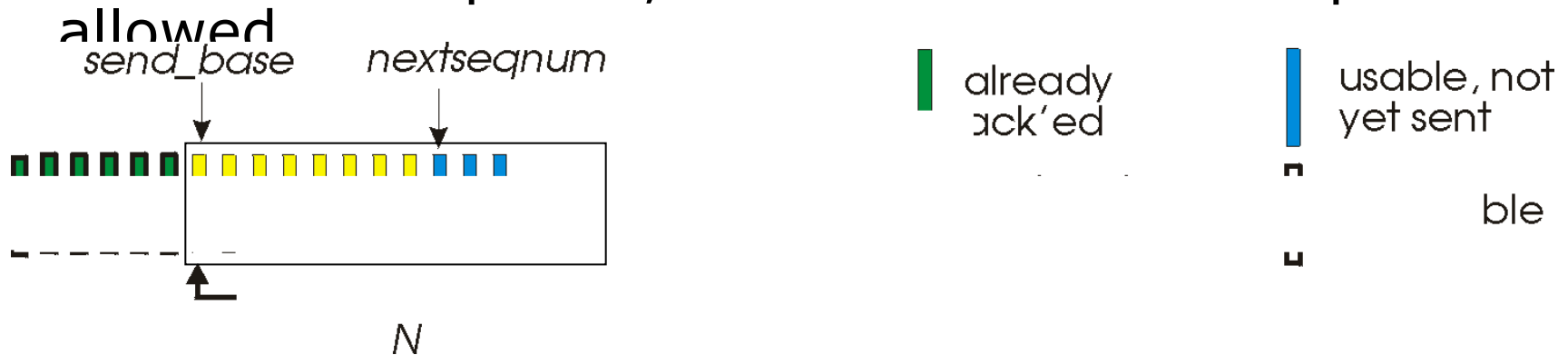
- ❖ sender can have up to N unack'ed ets in pipeline

al
t

- ❖ sender maintains timer for each unacked packet
 - when timer expires, retransmit only that unacked packet

Go-Back-N: sender

- ❖ k-bit seq # in pkt header
- ❖ “window” of up to N, consecutive unack’ed pkts

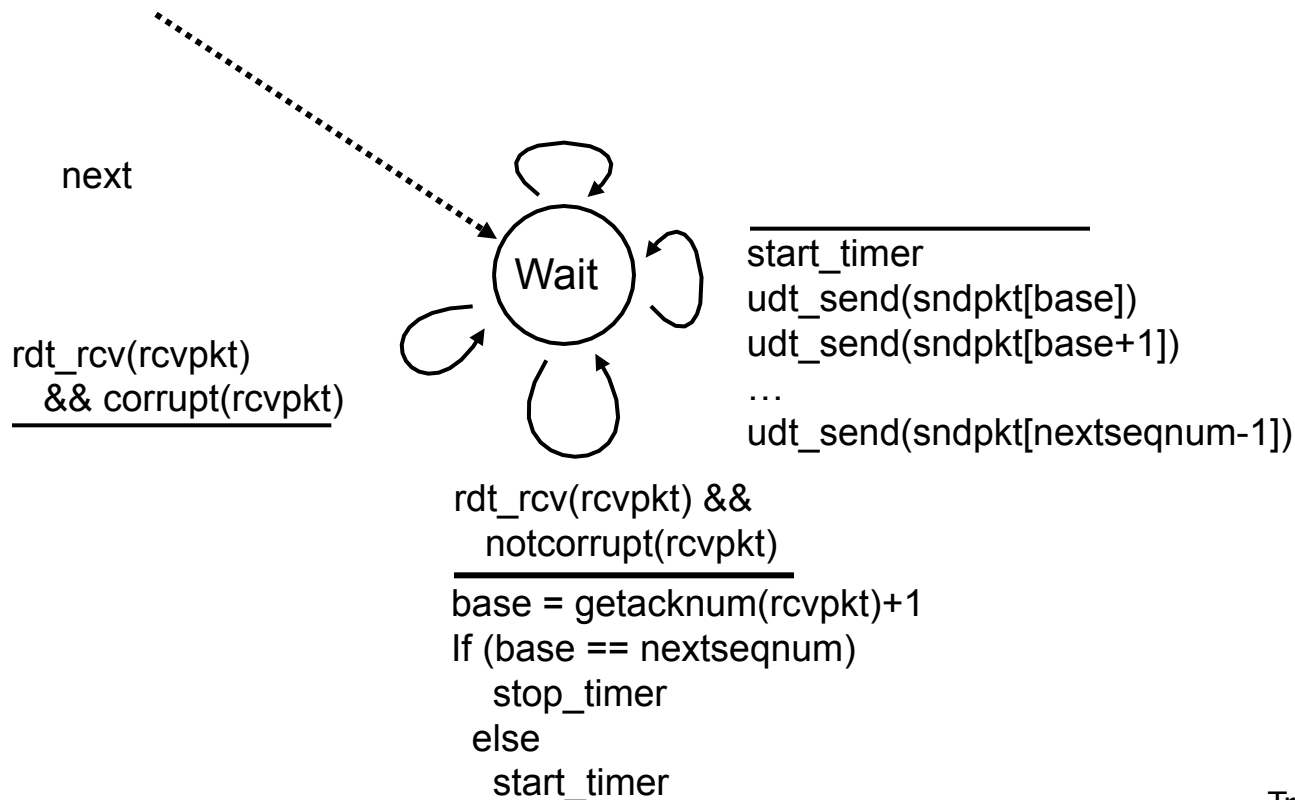


- ❖ ACK(n): ACKs all pkts up to, including seq # n -
 "cumulative ACK"
 - may receive duplicate ACKs (see receiver)
- ❖ timer for oldest in-flight pkt
- ❖ *timeout(n)*: retransmit packet n and all higher seq # pkts in window

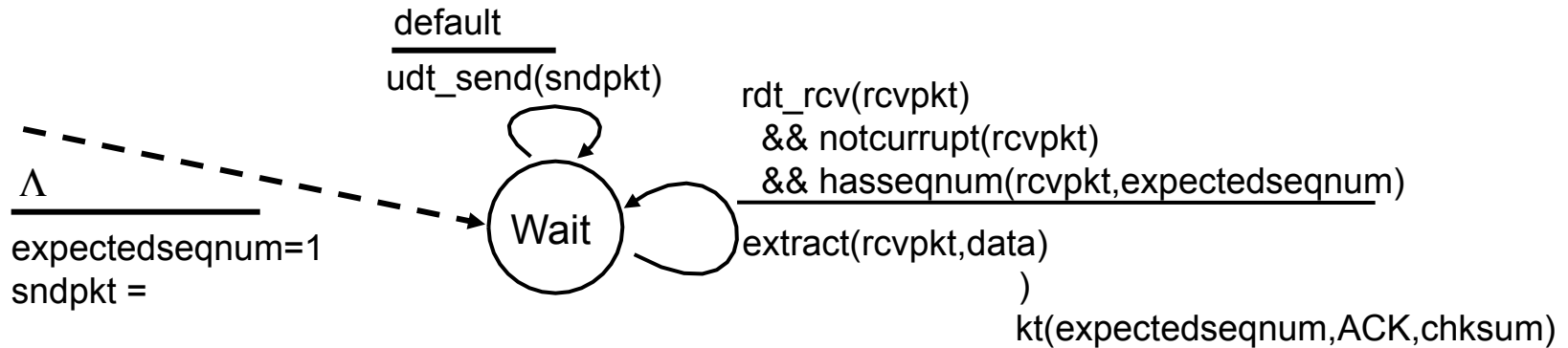
GBN: sender extended FSM

rdt_send(data)

```
if (nextseqnum < base+N) {  
    sndpkt[nextseqnum] = make_pkt(nextseqnum,data,chksum)  
    udt_send(sndpkt[nextseqnum])  
    if (base == nextseqnum)  
        start_timer
```



GBN: receiver extended FSM



- ACK-ectly-
received pkt with highest *in-order* seq #
- may generate duplicate ACKs
 - need only remember **expectedseqnum**
- ❖ out-of-order pkt:
- discard (don't buffer): *no receiver buffering!*
 - re-ACK pkt with highest in-order seq #

GBN in action

sender window (N=4)

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

0 1 2
0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8
0 1 2 3 4 5 6 7 8

sender

send pkt0
send pkt1
send pkt2
send pkt3
(

ignore duplicate ACK



pkt 2 timeout

send pkt2
send pkt3
send pkt4
send pkt5

receiver

receive pkt0, send ack0
receive pkt1, send ack1

, discard,
(re)send ack1
receive pkt5, discard,
(re)send ack1

rcv pkt2, deliver, send ack2
rcv pkt3, deliver, send ack3
rcv pkt4, deliver, send ack4
rcv pkt5, deliver, send ack5

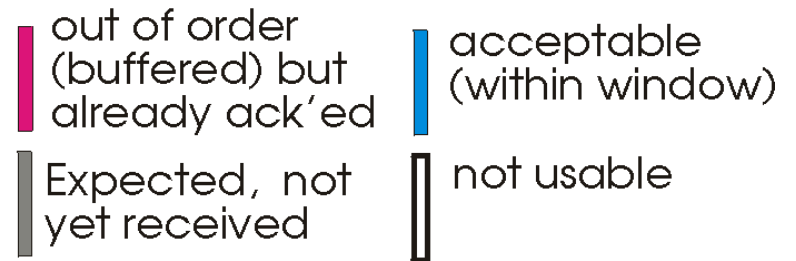
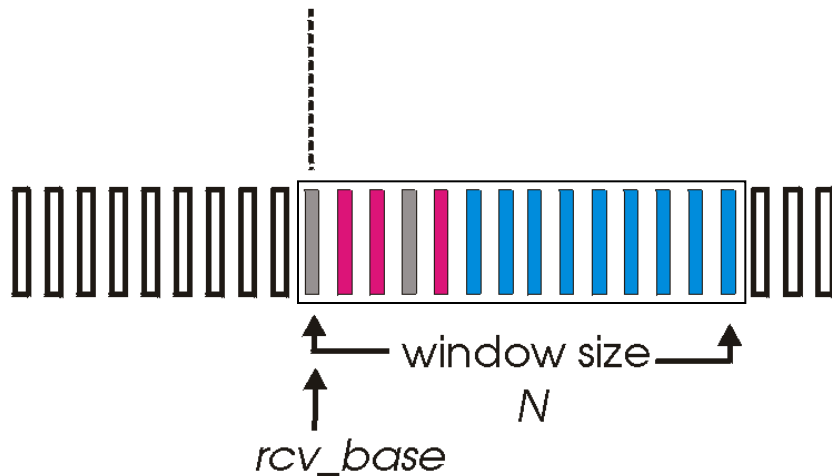
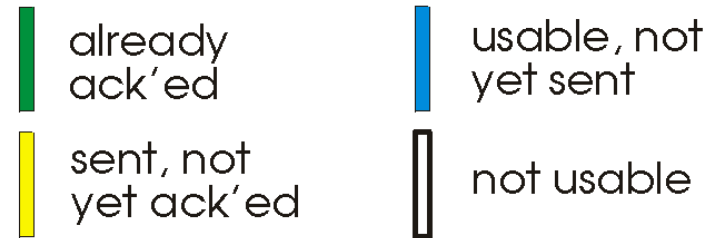
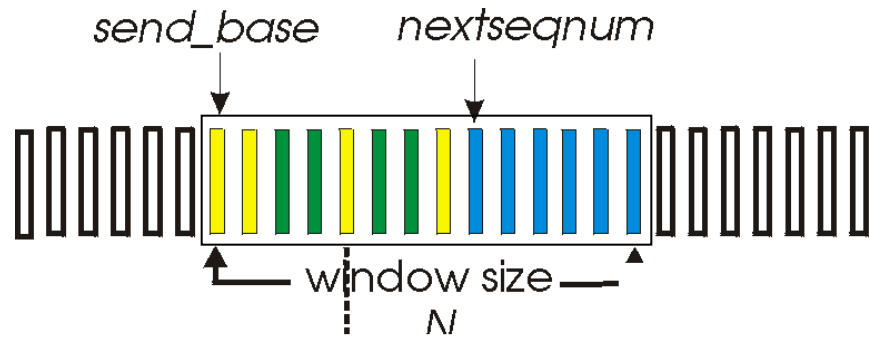
Selective repeat

- ❖ receiver *individually* acknowledges all correctly received pkts
 - buffers pkts, as needed, for eventual in-order deliver

ACK

- sender timer for each unACKed pkt
- ❖ sender window
 - N consecutive seq #'s
 - limits seq #'s of sent, unACKed pkts

Selective repeat: sender, receiver



(b) receiver view of sequence numbers

Selective repeat

sender

data from above:

- ❖ if next available seq # in window, s

- ❖ resen timer

ACK(n) in [sendbase, sendbase+N]:

- ❖ mark pkt n as received

- ❖ if n smallest unACKed pkt, advance window base to next

receiver

pkt n in [rcvbase, rcvbase+N-1]

- ❖ send ACK(n)

order pkts), advance window to next not-yet-received pkt

pkt n in [rcvbase-N, rcvbase-1]

- ❖ ACK(n)

otherwise:

- ❖ ignore

Selective repeat in action

sender window (N=4)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

sender

send pkt0

send pkt1

send pkt2

send pkt3

(

record ack3 arrived



pkt 2 timeout

send pkt2

record ack4 arrived

record ack5 arrived

receiver

receive pkt0, send ack0

receive pkt1, send ack1

, buffer,

send ack4

receive pkt5, buffer,

send ack5

rcv pkt2; deliver pkt2, pkt3, pkt4, pkt5; send ack2

Q: what happens when ack2 arrives?

Selective repeat dilemma

example:

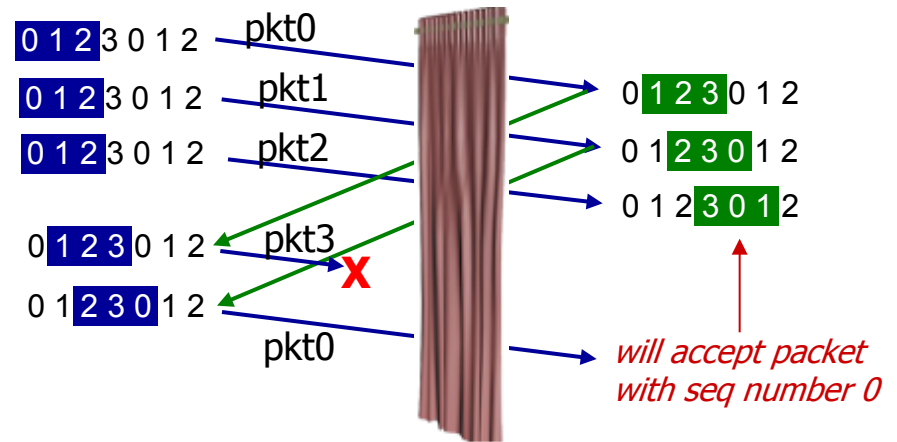
- ❖ seq #'s: 0, 1, 2, 3
- ❖ window size=3

- ❖ duplicate data accepted as new in (b)

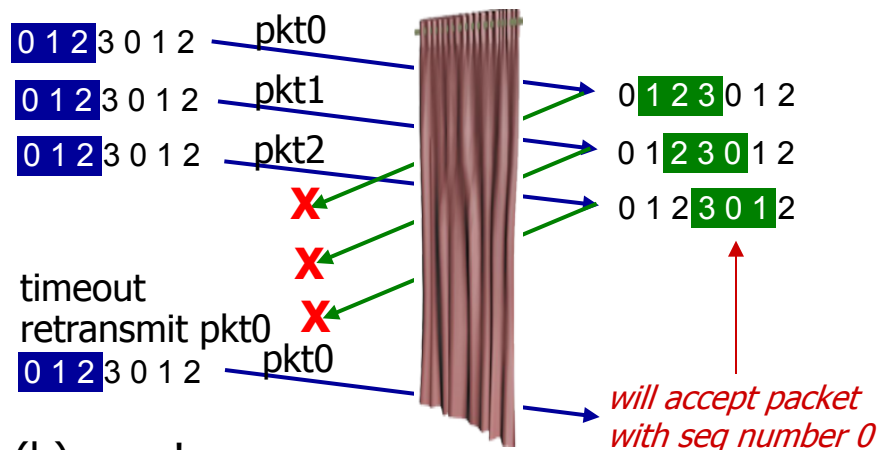
Q: what relationship between seq # size and window size to avoid problem in (b)?

sender window
(after receipt)

receiver window
(after receipt)



*n both cases!
rong!*



(b) oops!

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

TCP: Overview

RFCs: 793,1122,1323,

~~2018, 2581~~

❖ point-to-point:

- one sender, one receiver

- no boundaries"

❖ pipelined:

- TCP congestion and flow control set window size

❖ full duplex data:

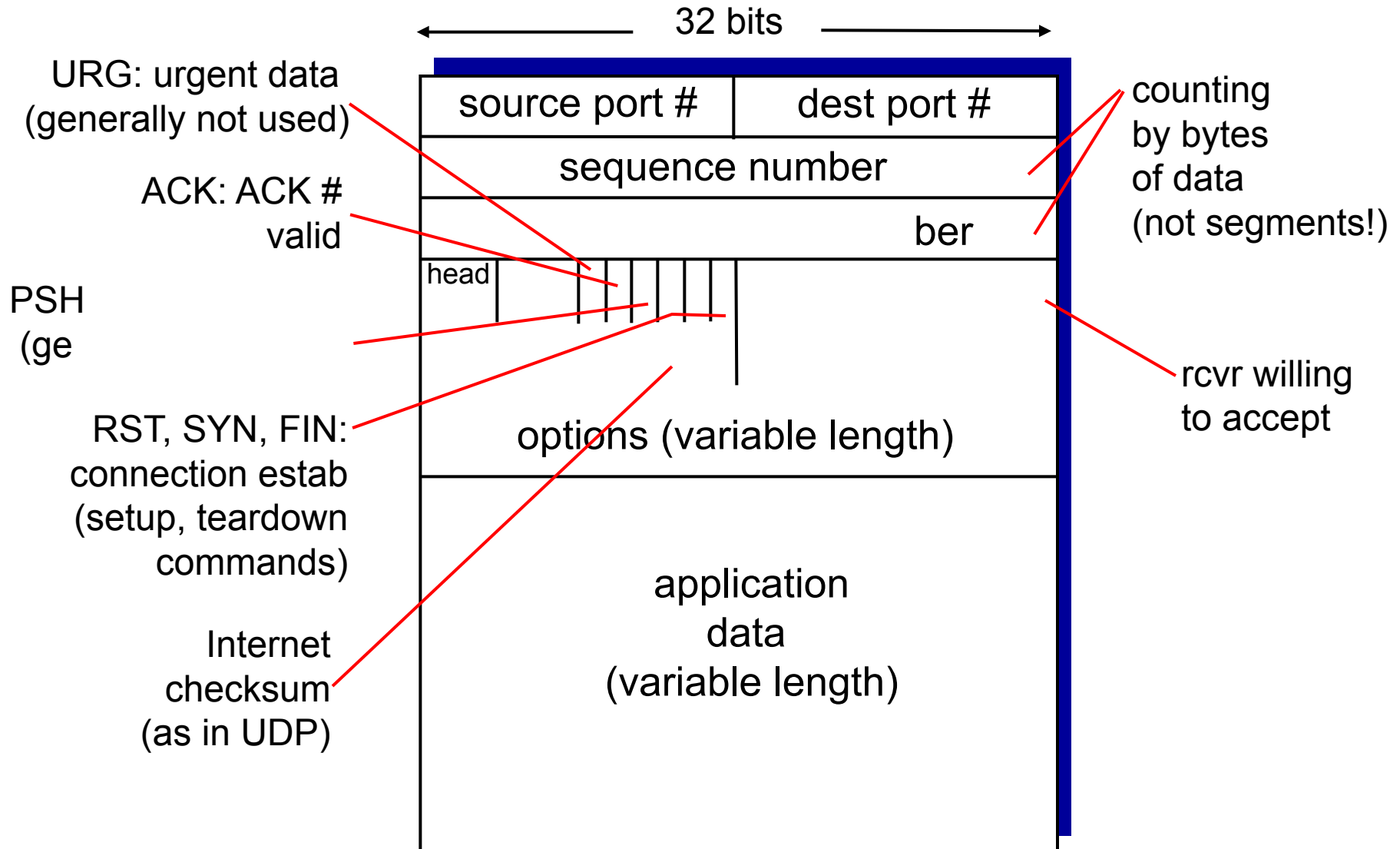
- bi-directional data flow in same connection

❖ connection-oriented:

- handshaking (exchange of control msgs) initializes sender, receiver state before data exchange

❖ flow controlled:

TCP segment structure



TCP seq. numbers, ACKs

sequence numbers:

- byte stream
“number” of first byte in segmen

a

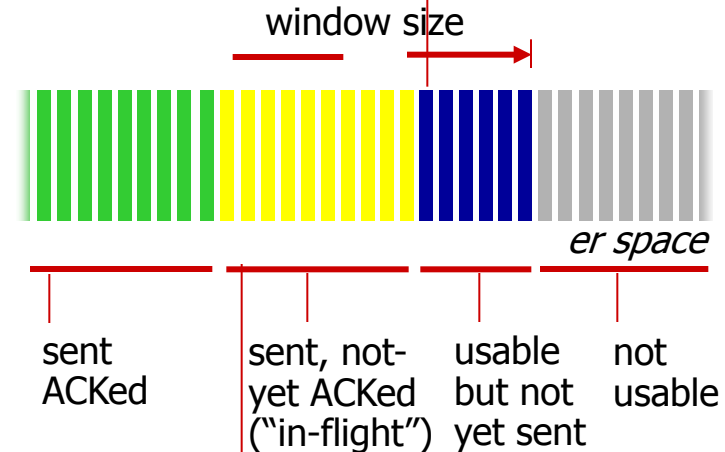
- seq #
expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A:** TCP spec doesn't say, - up to implementor

outgoing segment from sender

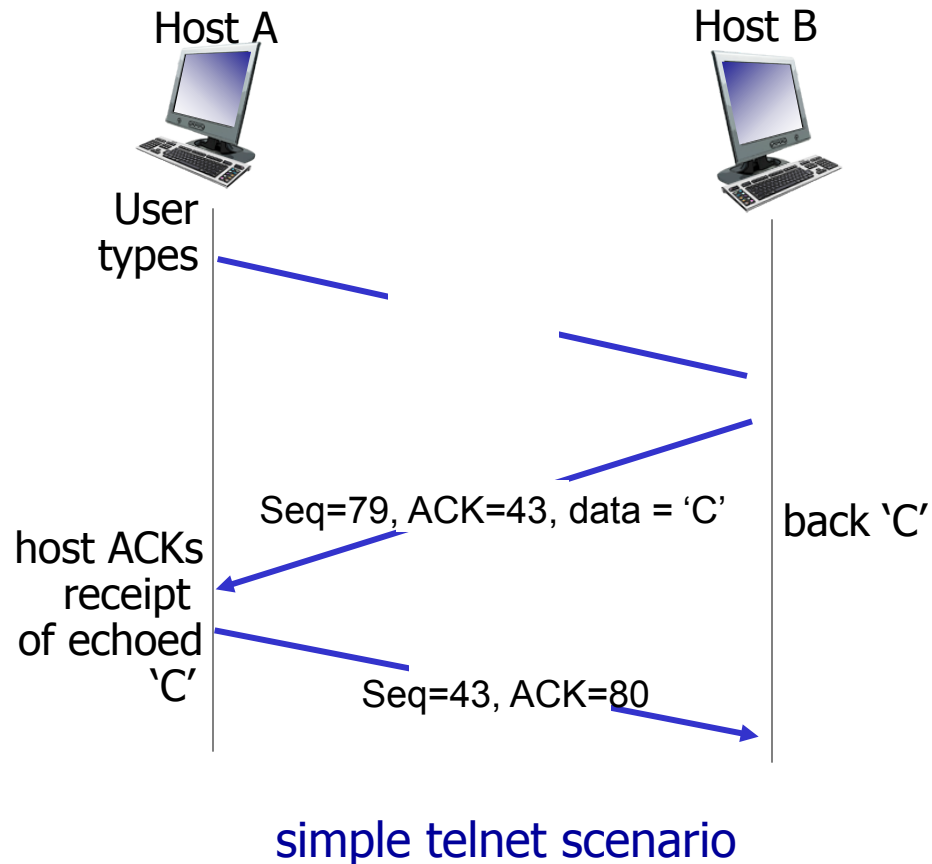
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



incoming segment to sender

source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer

TCP seq. numbers, ACKs



TCP round trip time, timeout

Q: how to set
TCP timeout
value?

- ❖ *too s*
premature
timeout,
unnecessary
retransmissions
- ❖ *too long:* slow
reaction to
segment loss

Q: how to estimate
RTT?

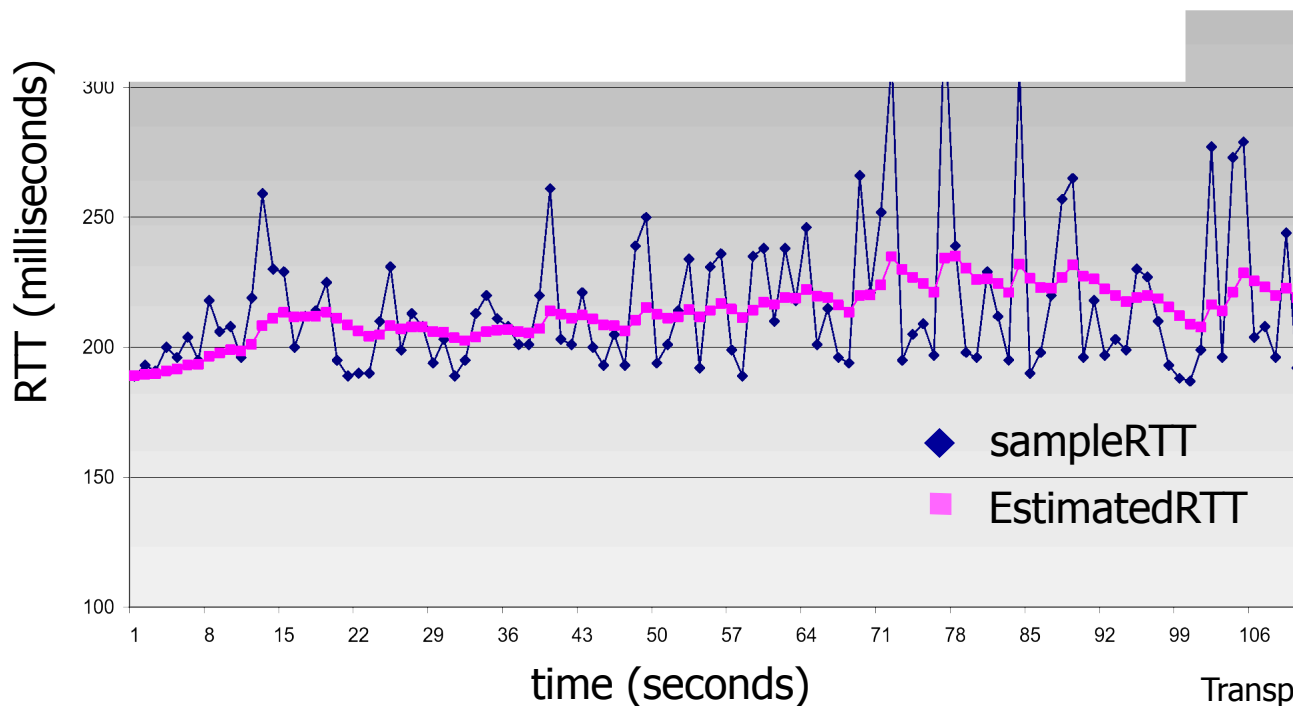
- ❖ **SampleRTT:** measured
from segment

- retransmissions
- ❖ **SampleRTT** will vary,
want estimated RTT
“smoother”
 - average several
recent
measurements, not
just current
SampleRTT

TCP round trip time, timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❖ exponential weighted moving average
- ❖ influence of past sample decreases exponential



TCP round trip time, timeout

❖ **timeout interval:** `EstimatedRTT` plus “safety margin”

- large variation in margin

larger safety

Estim

0.25)

TimeoutInterval = EstimatedRTT + 4*DevRTT



↑
estimated RTT

↑
“safety margin”

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

TCP reliable data transfer

- ❖ TCP creates rdt service on top of IP's unreliable
 - cu
 - single retransmission timer
- ❖ retransmissions triggered by:
 - timeout events
 - duplicate acks
- ignore duplicate acks
- ignore flow control, congestion control

TCP sender events:

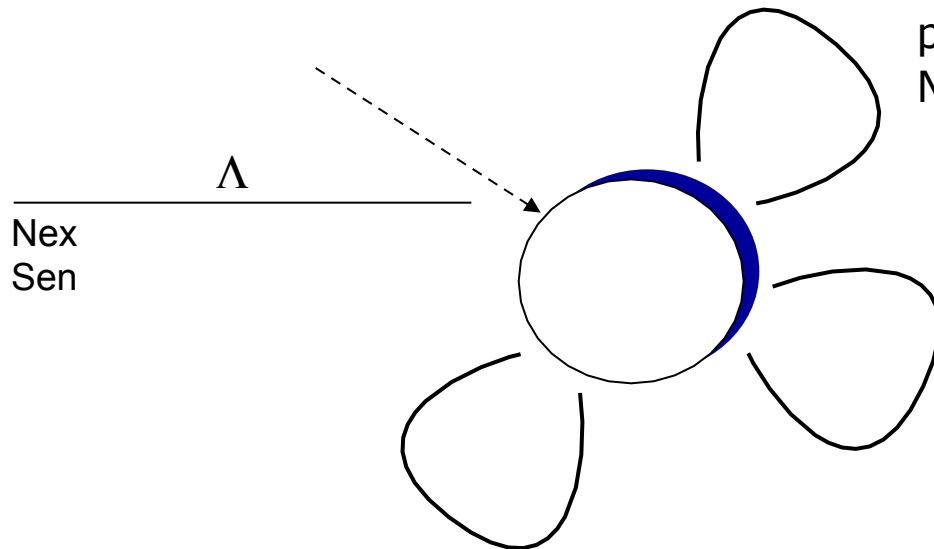
data rcvd from app: *timeout:*

- ❖ create segment with seq #
- ❖ seq # is byte
- ❖ retransmit segment that d timeout

segm

- ❖ start timer if not already running
 - think of timer as for oldest unacked segment
 - expiration interval: `TimeoutInterval`
- ❖ if ack acknowledges previously unacked segments
 - update what is known to be ACKed
 - start timer if there

TCP sender (simplified)



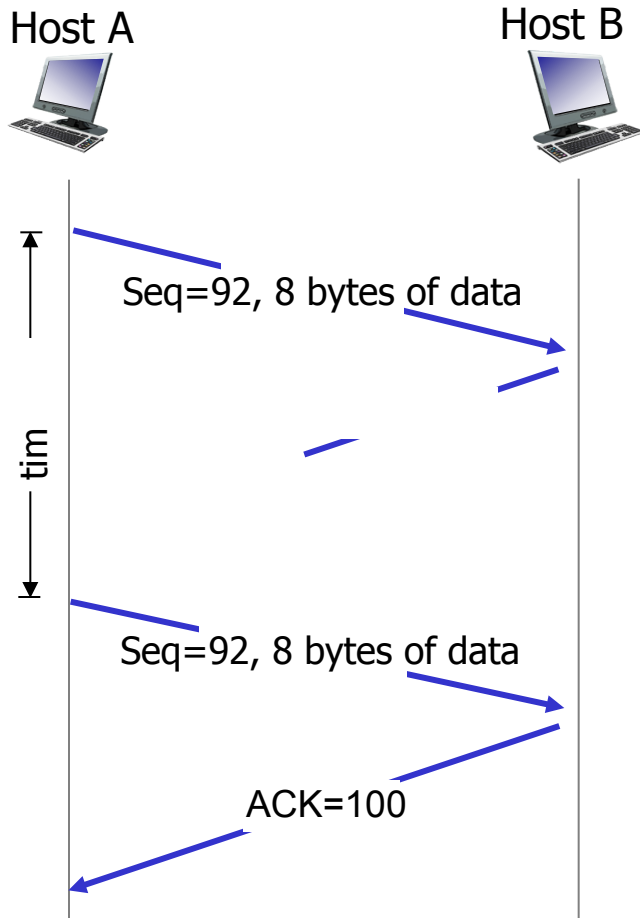
data received from application above
create segment, seq. #: NextSeqNum
pass segment to IP (i.e., "send")
NextSeqNum = NextSeqNum + length(data)
currently not running
r

retransmit not-yet-acked segment
with smallest seq. #
start timer

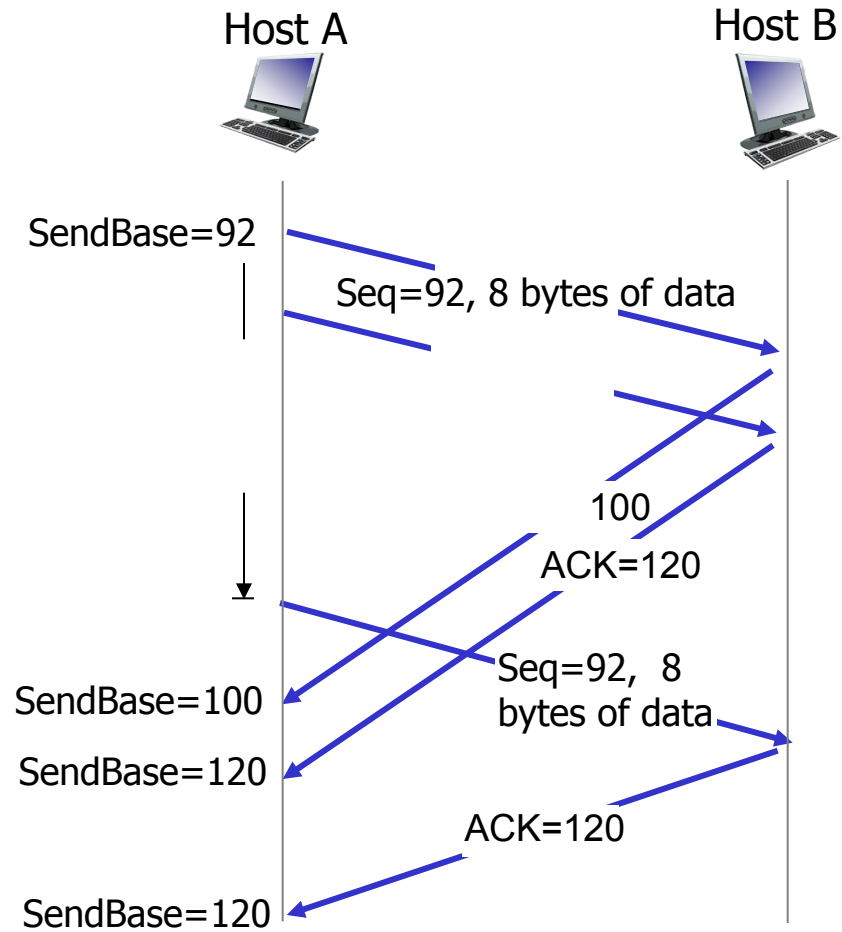
ACK received, with ACK field value y

```
if (y > SendBase) {  
    SendBase = y  
    /* SendBase-1: last cumulatively ACKed byte */  
    if (there are currently not-yet-acked segments)  
        start timer  
    else stop timer  
}
```

TCP: retransmission scenarios

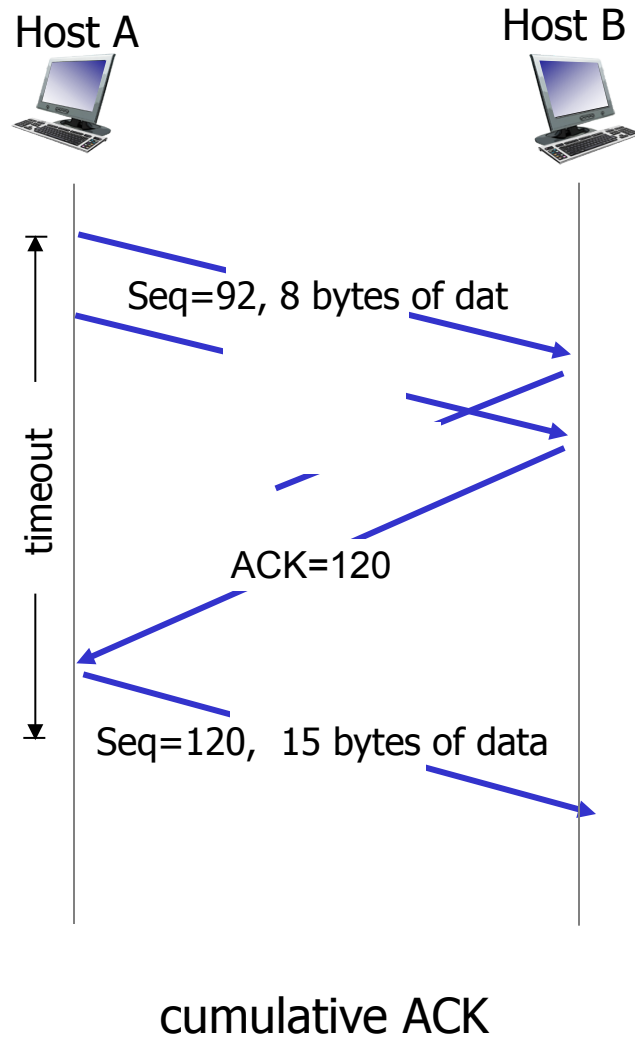


lost ACK scenario



premature timeout

TCP: retransmission scenarios



TCP ACK generation [RFC 1122, RFC 2581]

<i>event at receiver</i>	<i>TCP receiver action</i>
arrival of in-order segment with expected seq #. All data	delayed ACK. Wait up to 500ms ment. If no next segment,
arrival of i expected segment has ACK pending	mulative r segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send <i>duplicate ACK</i> , indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

TCP fast retransmit

- ❖ time-out period often relatively long:
 - long delay be
- ❖ detect segments via duplicate ACKs.
 - sender often sends many segments back-to-back
 - if segment is lost, there will likely be many duplicate ACKs

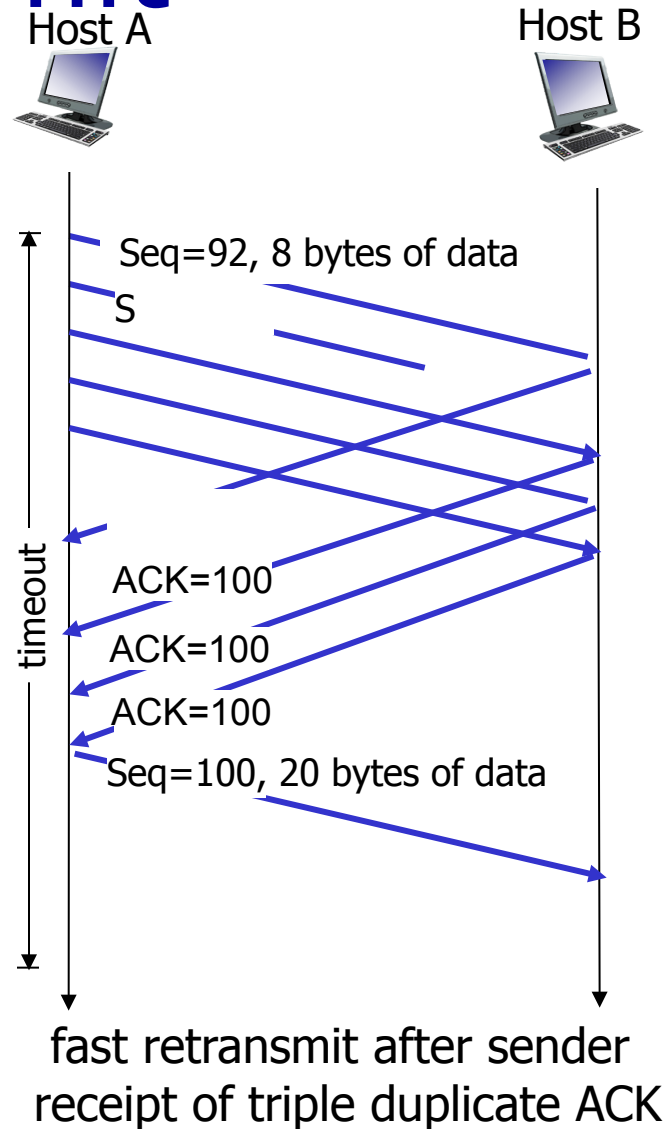
TCP fast retransmit

if sender receives
Ks for same

unacked segment
with smallest seq
#

- likely that unacked segment lost, so don't wait for timeout

TCP fast ~~retransmit~~



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

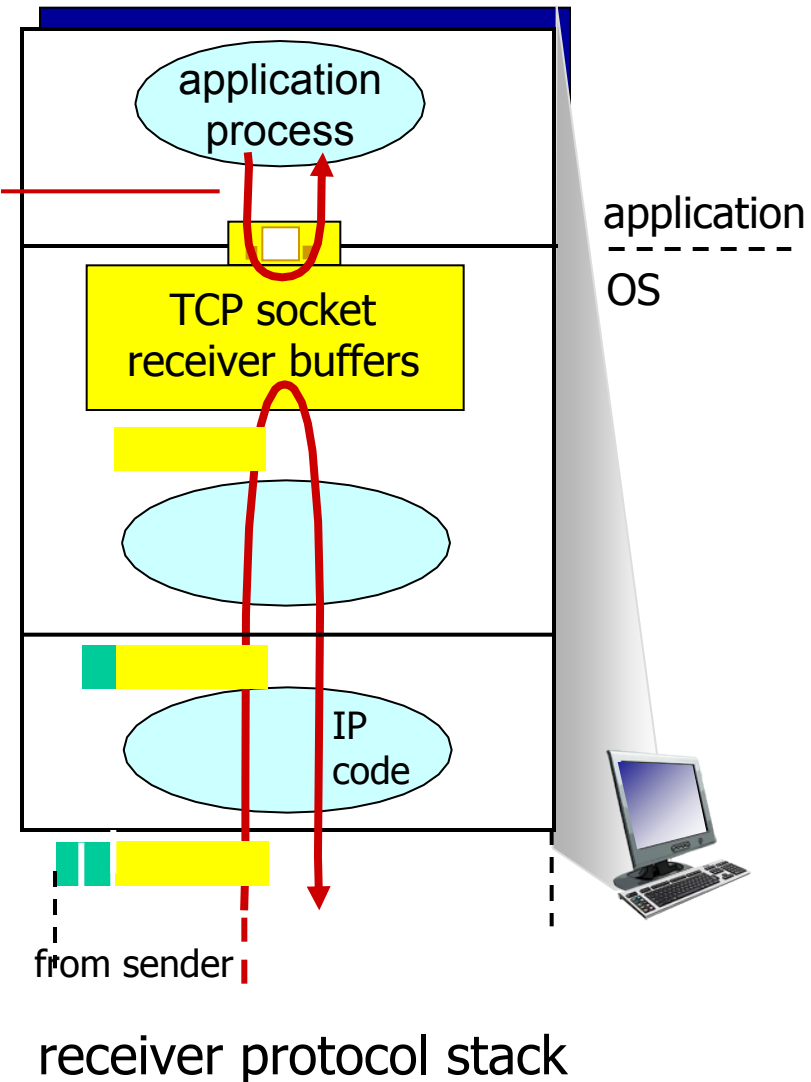
3.6 principles of
congestion control

3.7 TCP congestion
control

TCP flow control

application may
remove data from
TCP socket buffers

flow control
receiver controls sender,
so sender won't overflow
receiver's buffer by
transmitting too much,
too fast

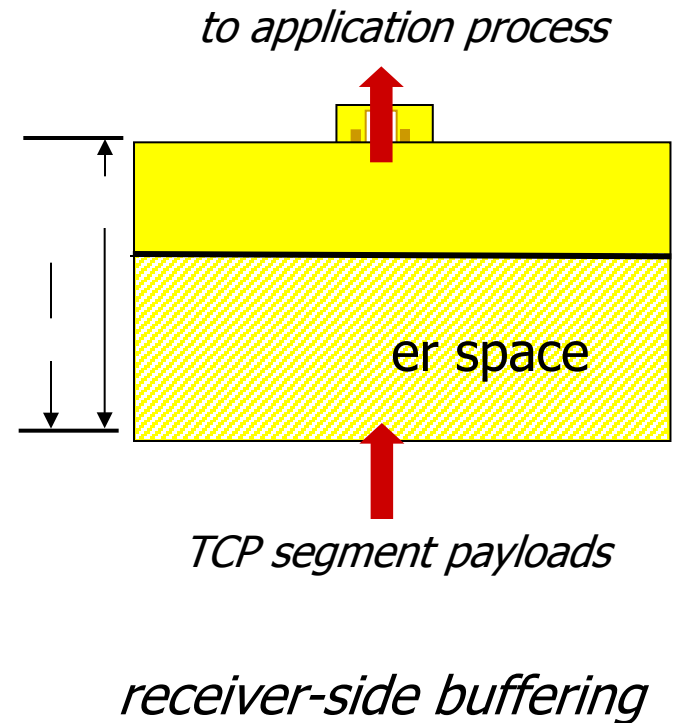


TCP flow control

- ❖ receiver “advertises” free buffer space by including **rwnd** value in TCP header of

- **RcvB**
socket options (typical default is 4096 bytes)
- many operating systems autoadjust **RcvBuffer**

- ❖ sender limits amount of unacked (“in-flight”) data to receiver’s **rwnd** value
- ❖ guarantees receive buffer will not



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

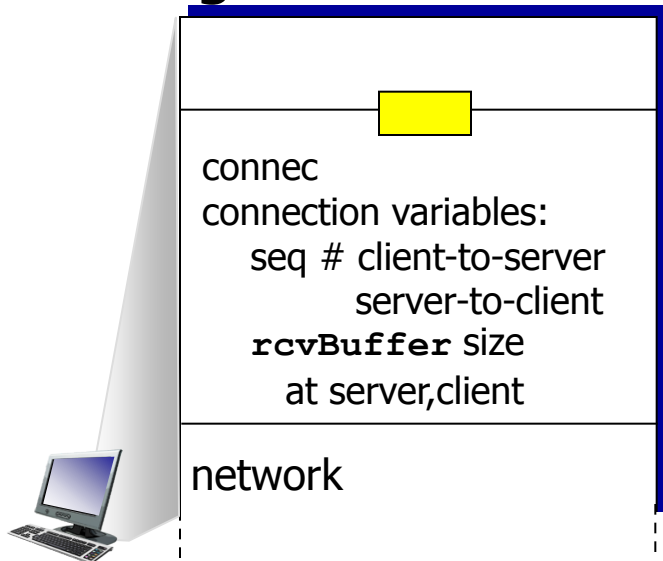
3.6 principles of
congestion control

3.7 TCP congestion
control

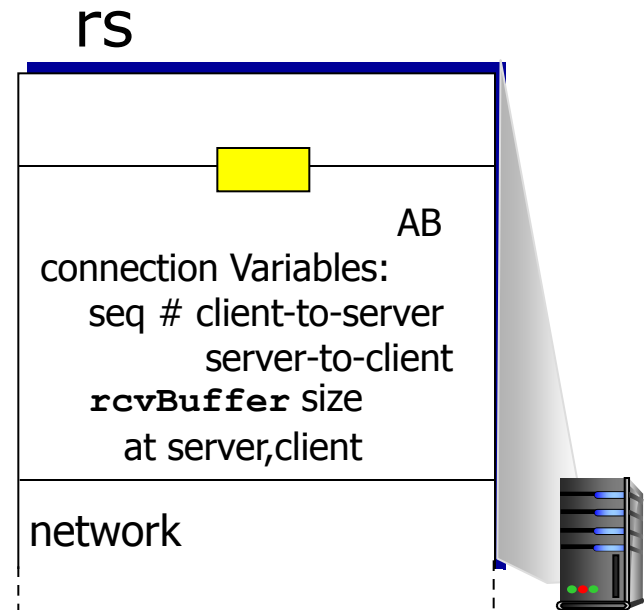
Connection Management

before exchanging data, sender/receiver
"handshake":

- ❖ agree to establish connection (each knowing the other willing to establish connection)
- ❖ agree on conn



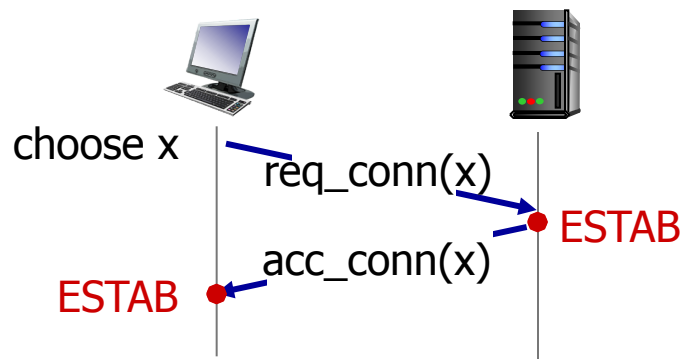
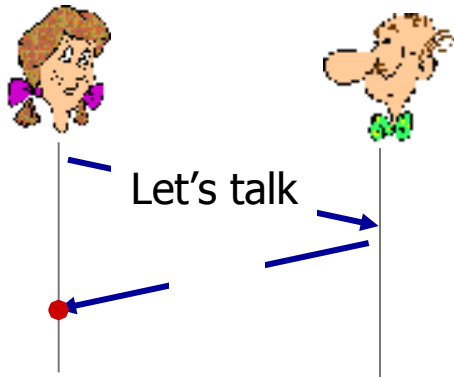
```
Socket clientSocket =  
    newSocket("hostname", "port  
    number");
```



```
Socket connectionSocket =  
    welcomeSocket.accept();
```

Agreeing to establish a connection

2-way handshake:



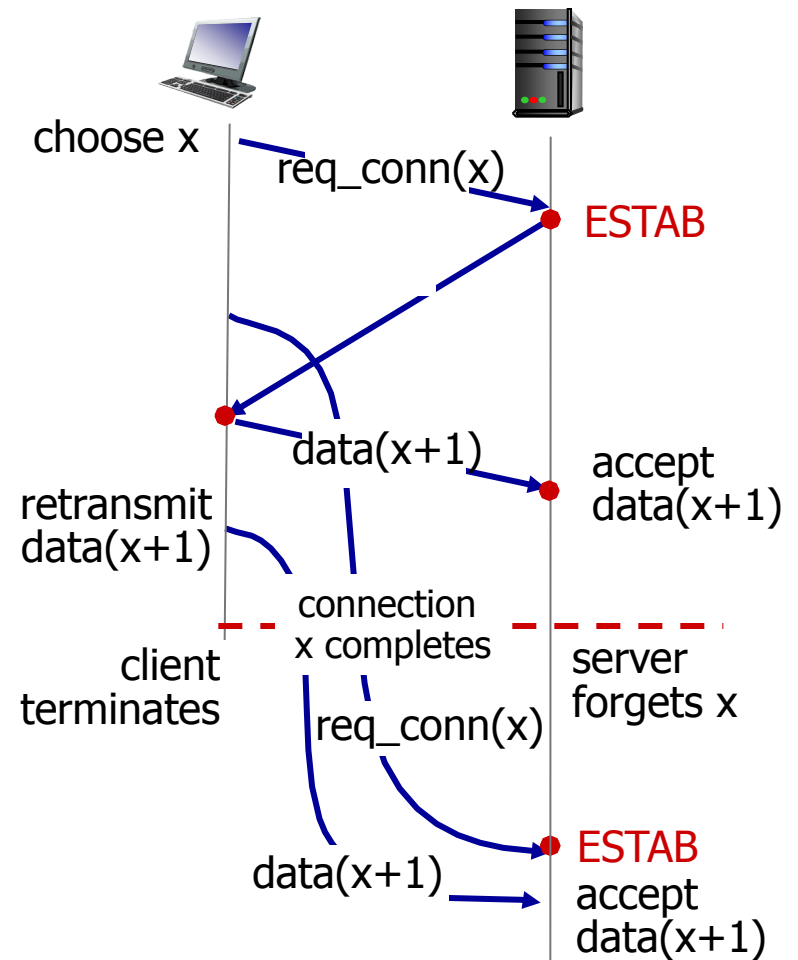
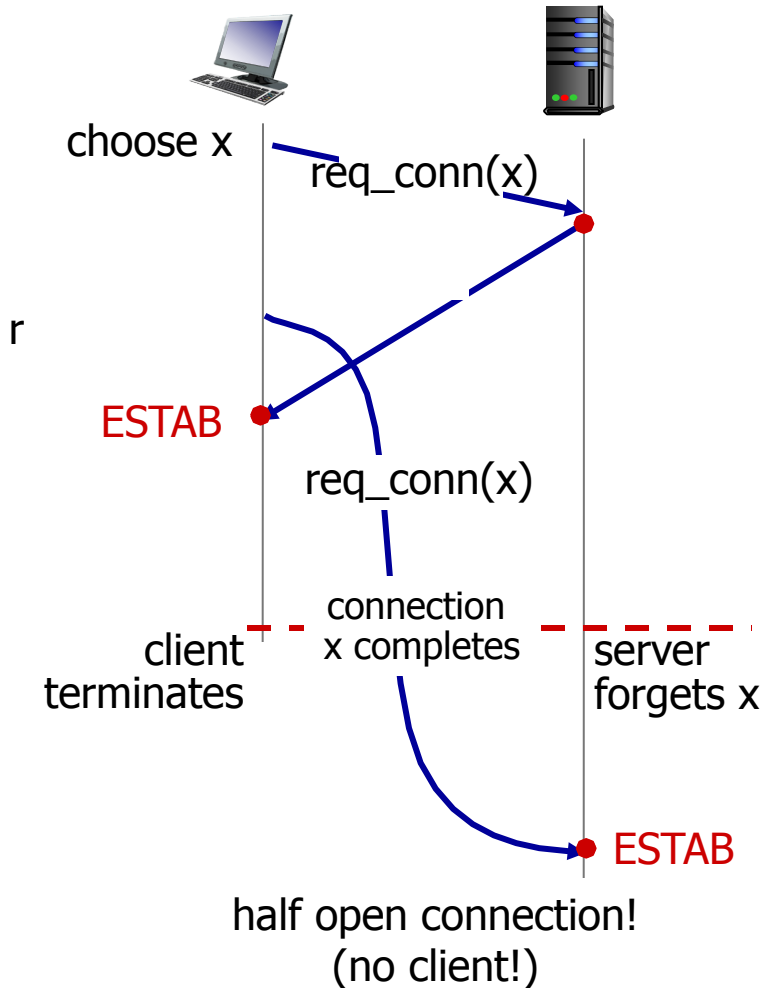
Q: will 2-way handshake always in network?

req_conn(x)) due to message loss

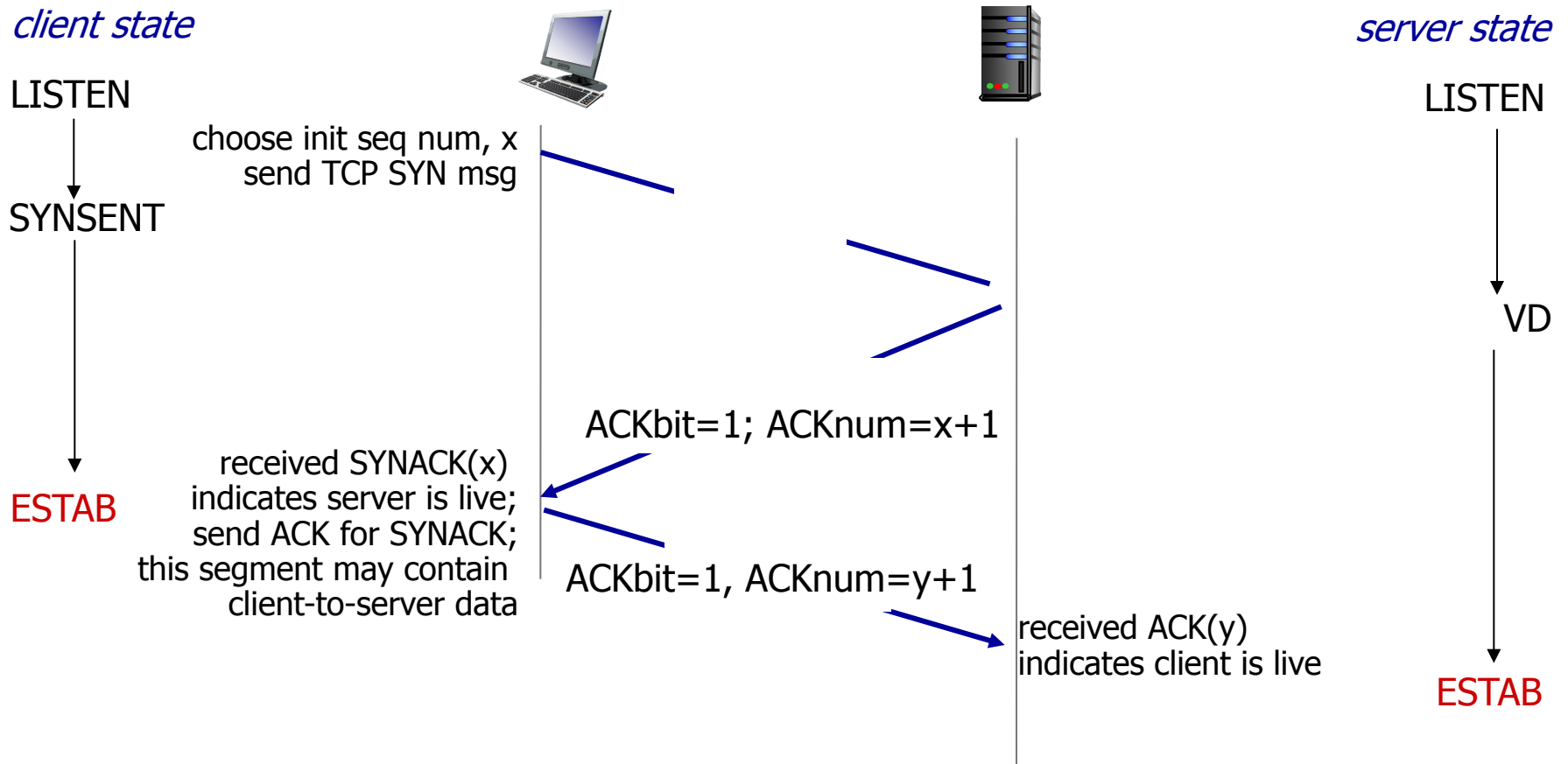
- ❖ message reordering
- ❖ can't "see" other side

Agreeing to establish a connection

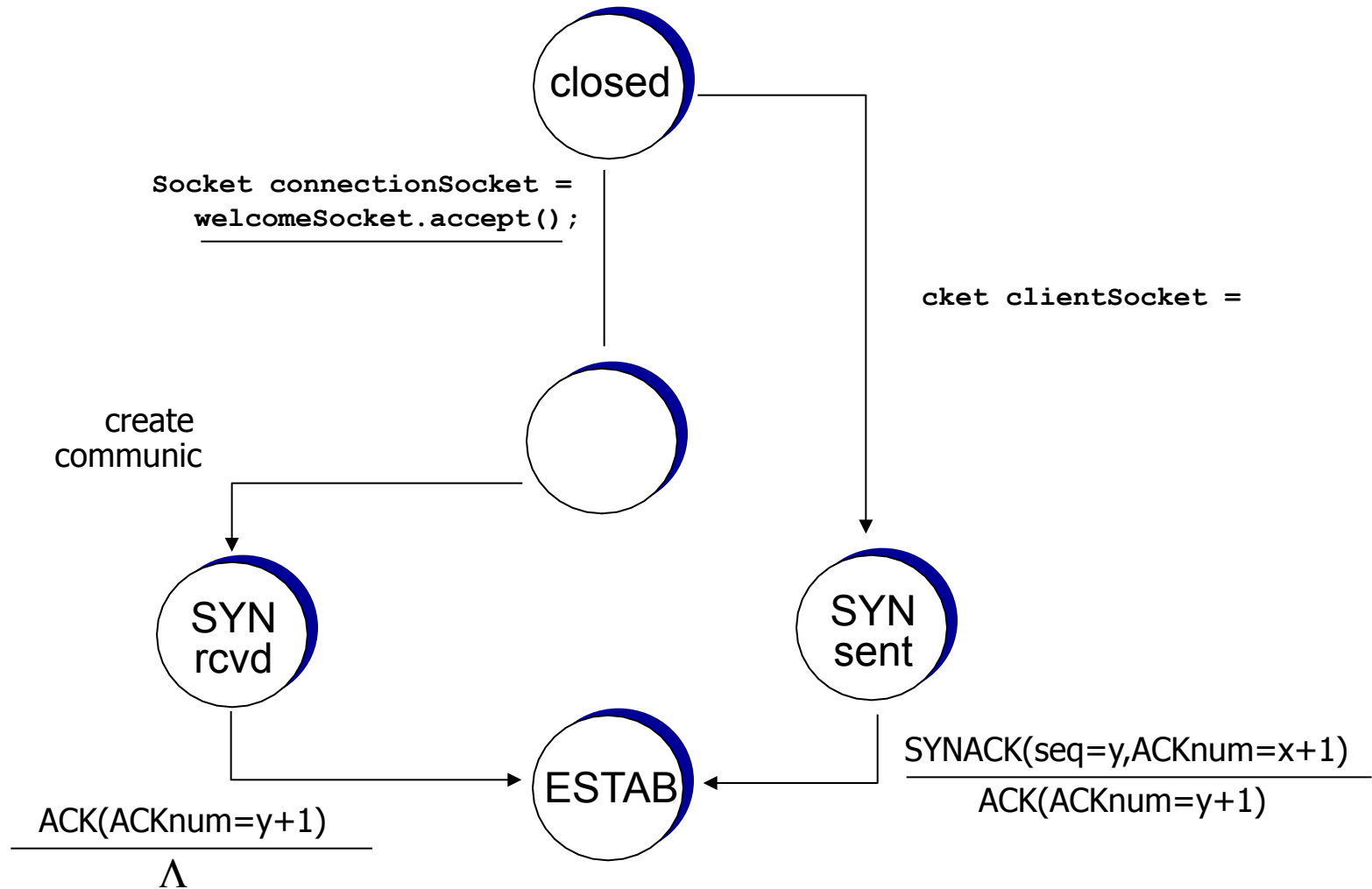
2-way handshake failure scenarios:



TCP 3-way handshake



TCP 3-way handshake: FSM



TCP: closing a connection

- ❖ client, server each close their side of connection
 - send TCP segment with FIN bit = 1
- ❖ respond to rith ACK
- ❖ simultaneous FIN exchanges can be handled

TCP: closing a connection

client state

ESTAB

`clientSocket.close()`

FIN_WAIT_1

can no longer
send but can

F

TIMED_WAIT

timed wait
for $2 * \text{max}$
segment lifetime

CLOSED



server state

ESTAB

LAST_ACK

CLOSED

FINbit=1, seq=y

ACKbit=1; ACKnum=y+1

can no longer
send data

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

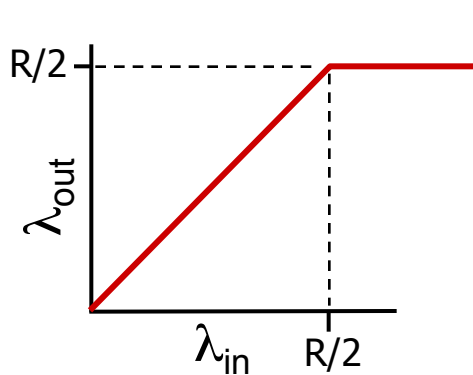
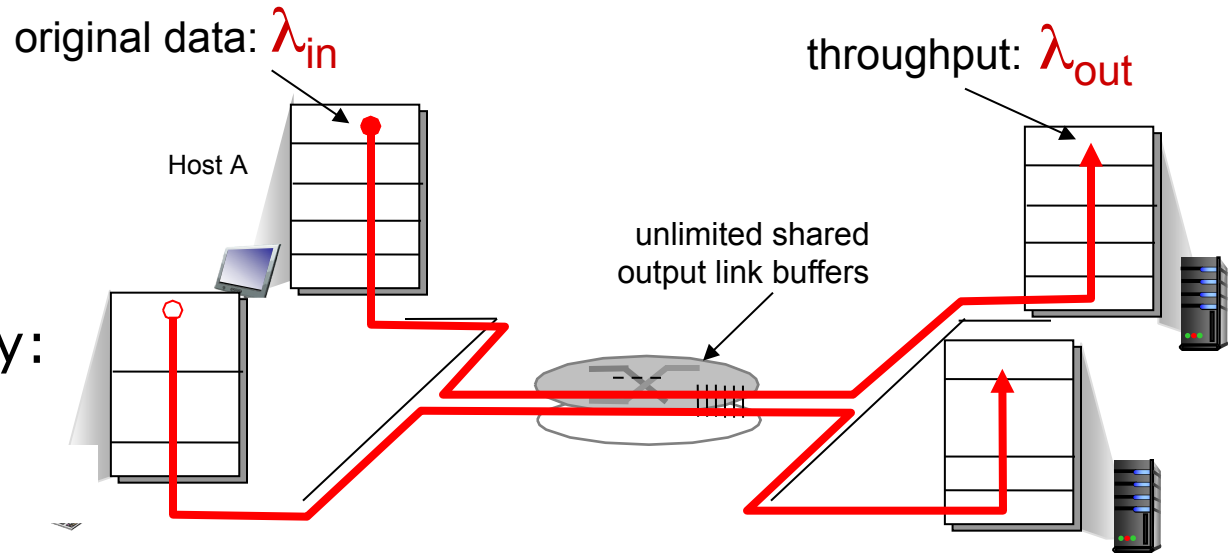
Principles of congestion control

congestion:

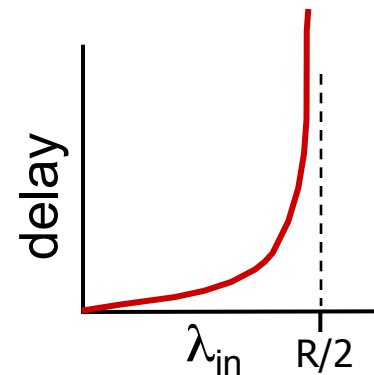
- ❖ informally: “too many sources sending too much dat *network* to
- ❖
- ❖ manifestations:
 - lost packets (buffer overflow at routers)
 - long delays (queueing in router buffers)
- ❖ a top-10 problem!

Causes/costs of congestion: scenario 1

- ❖ two senders, two receivers
- ❖ one router, infinite buffers
- ❖ output link capacity:
- ❖



- ❖ maximum per-connection throughput: $R/2$

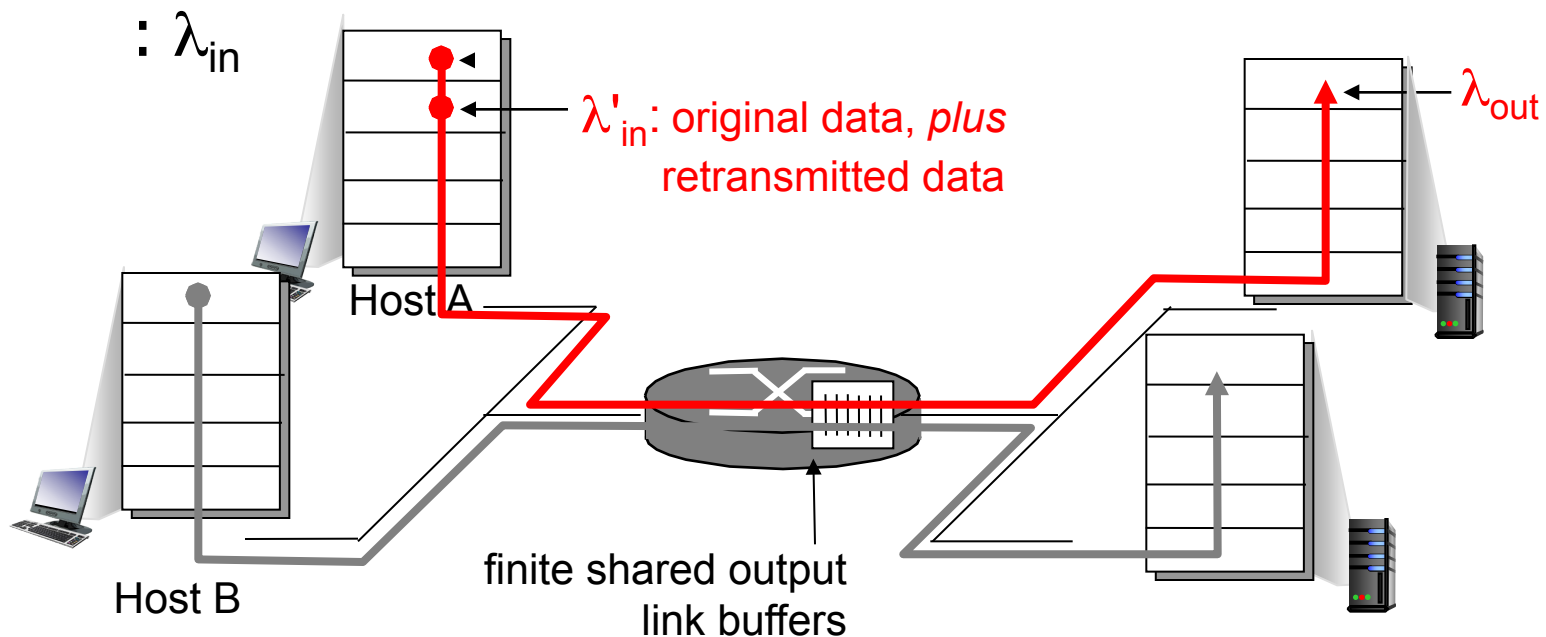


- ❖ large delays as arrival rate, λ_{in} , approaches capacity

Causes/costs of congestion:

scenario 2

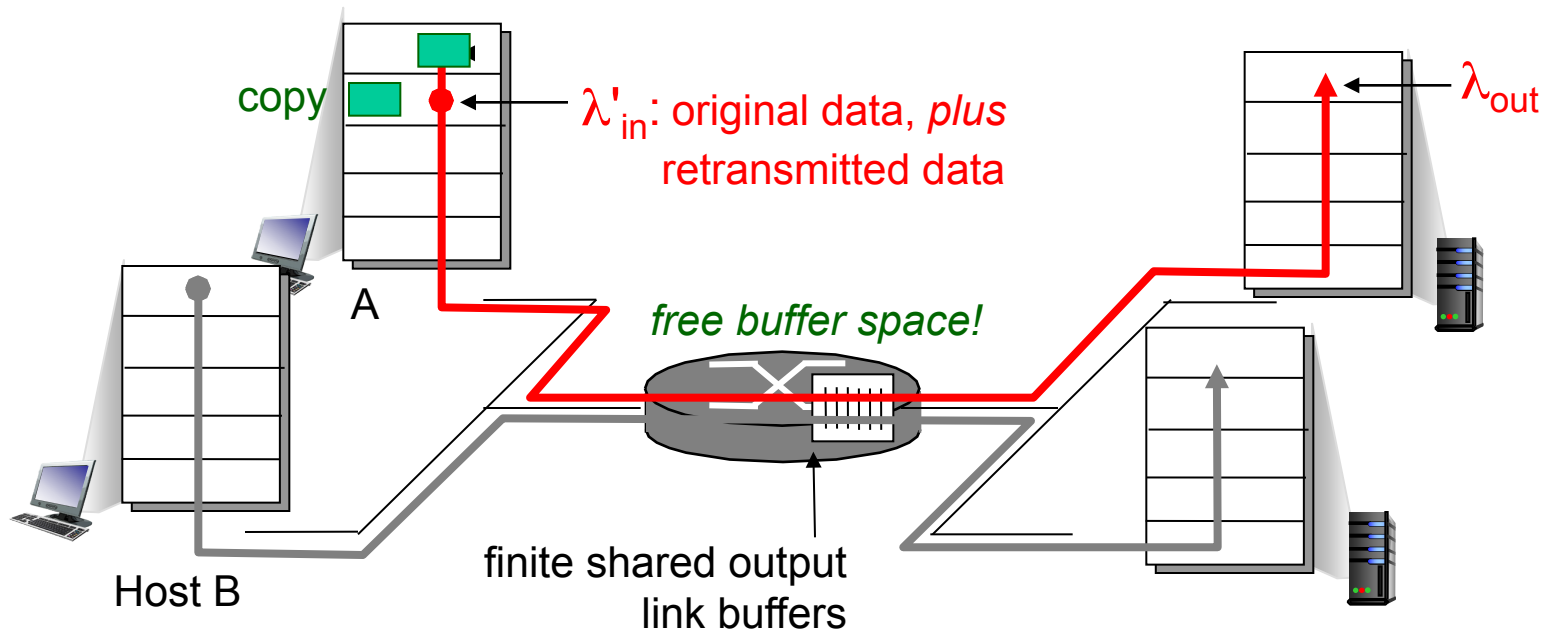
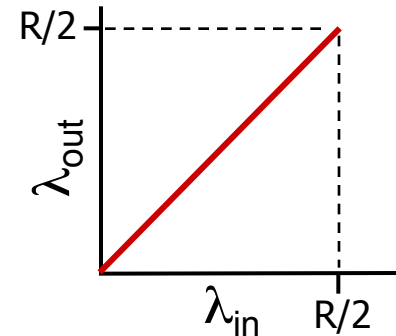
- ❖ one router, *finite* buffers
- ❖ sender retransmission of timed-out packet
 - application-layer



Causes/costs of congestion: ~~scenario 2~~

idealization: perfect knowledge

- ❖ sender sends only when router b available



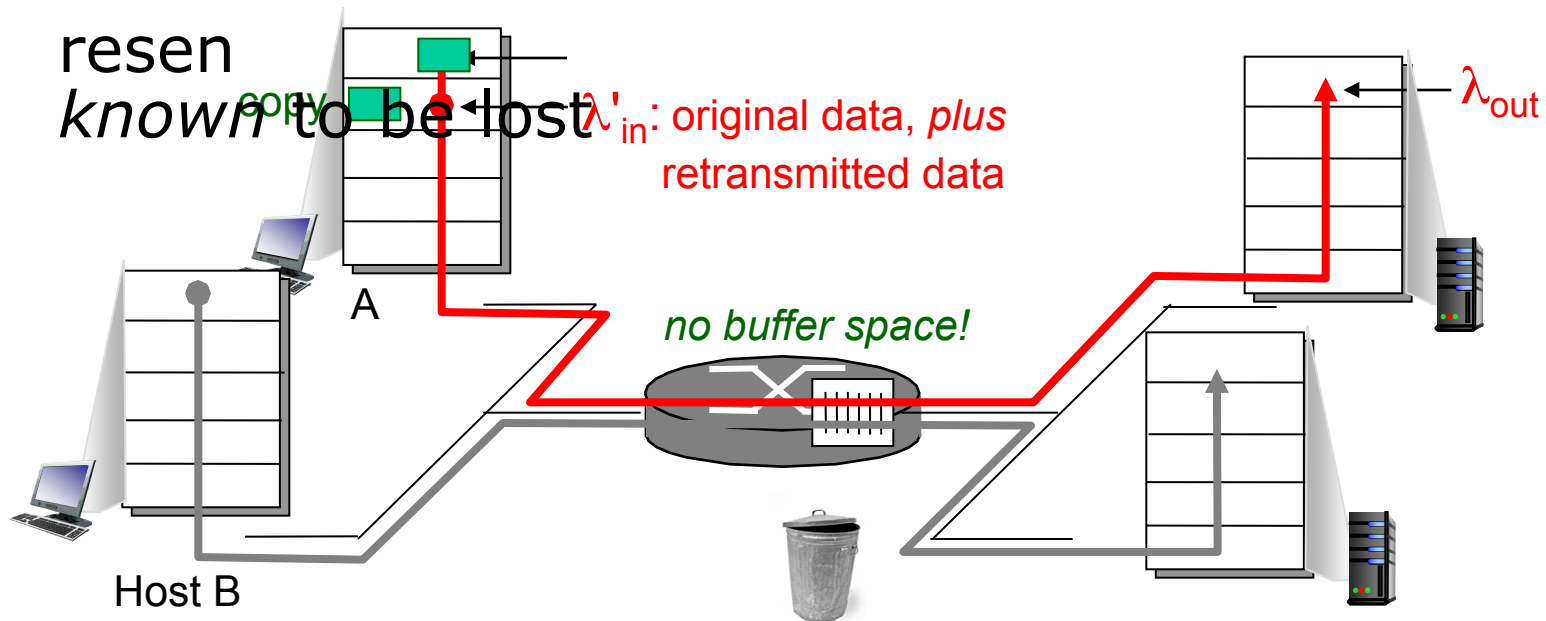
Causes/costs of congestion:

~~scenario 2~~

Idealization:

known loss

packets can be
lost, dropped at
router due to f



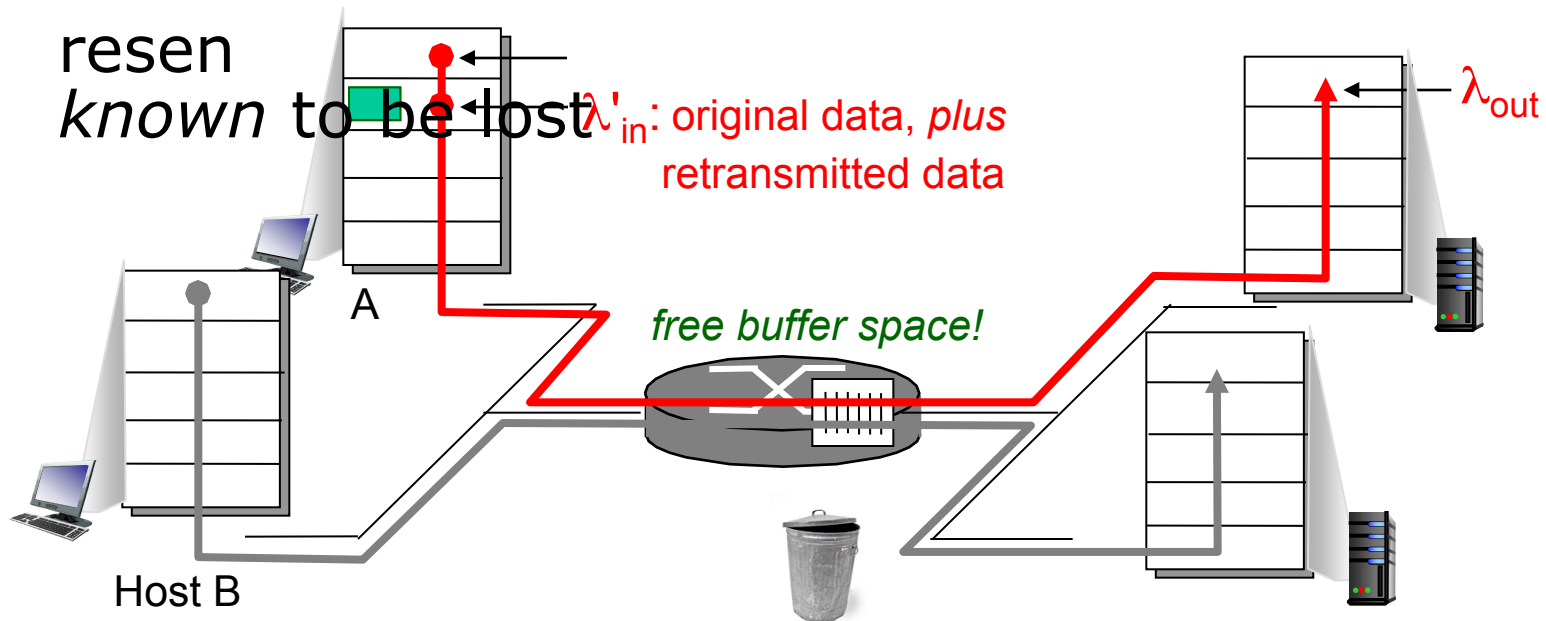
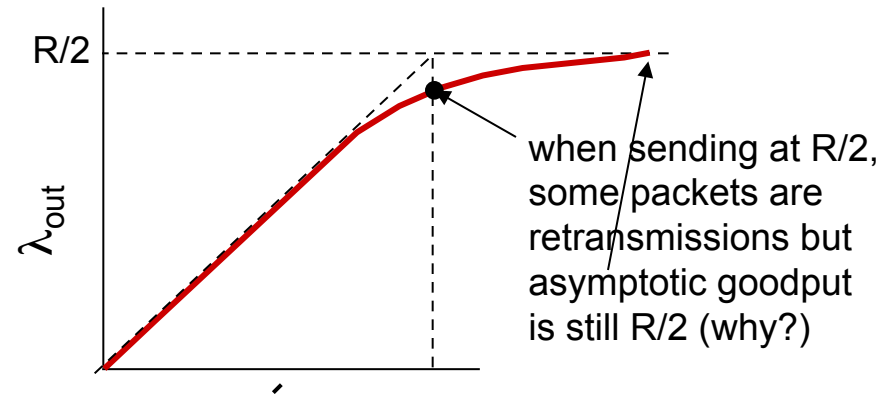
Causes/costs of congestion:

~~scenario 2~~

Idealization:

known loss

packets can be lost, dropped at router due to f

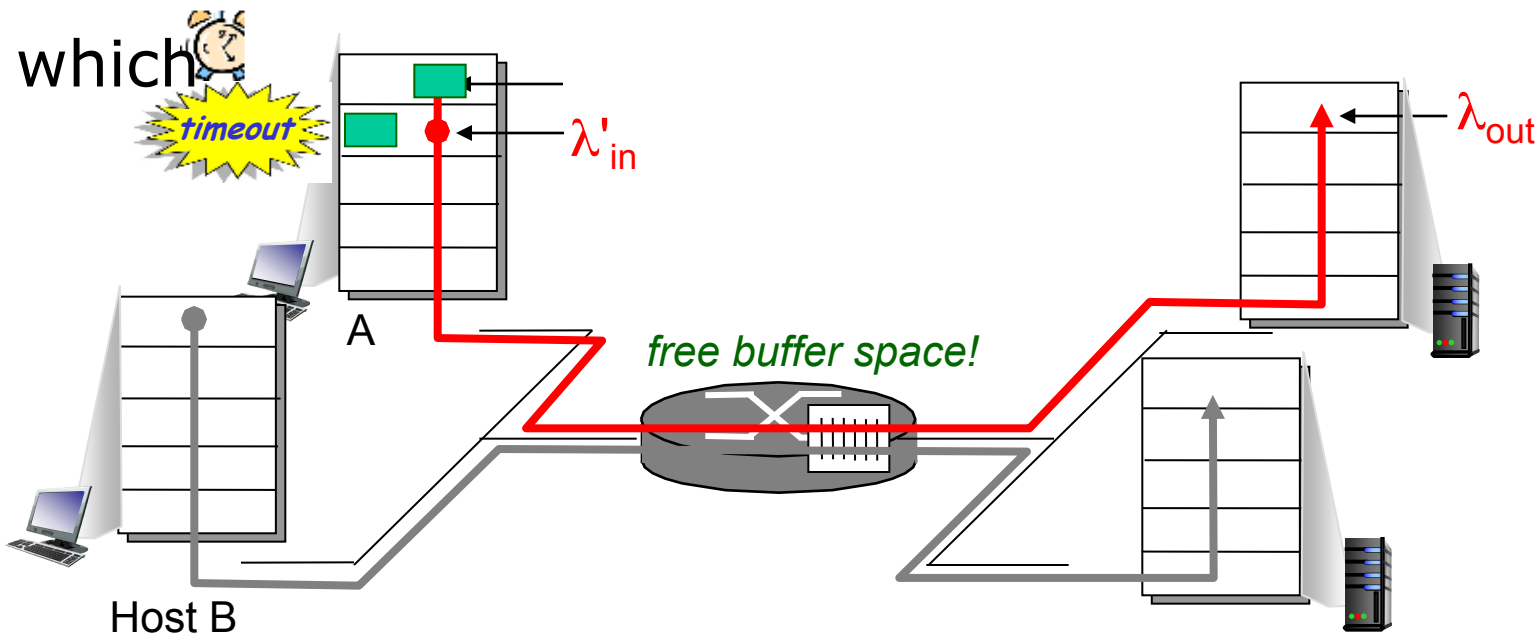
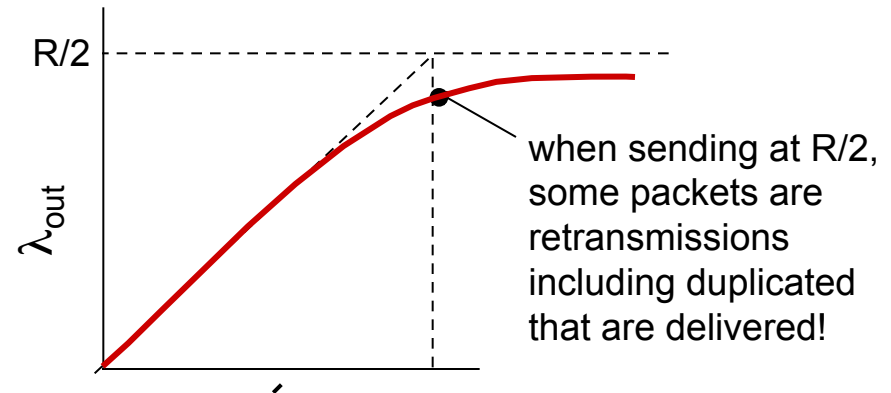


Causes/costs of congestion:

~~scenario 2~~

Realistic: duplicates

- ❖ packets can be lost, dropped at router due to full buffers
- ❖ sender times out

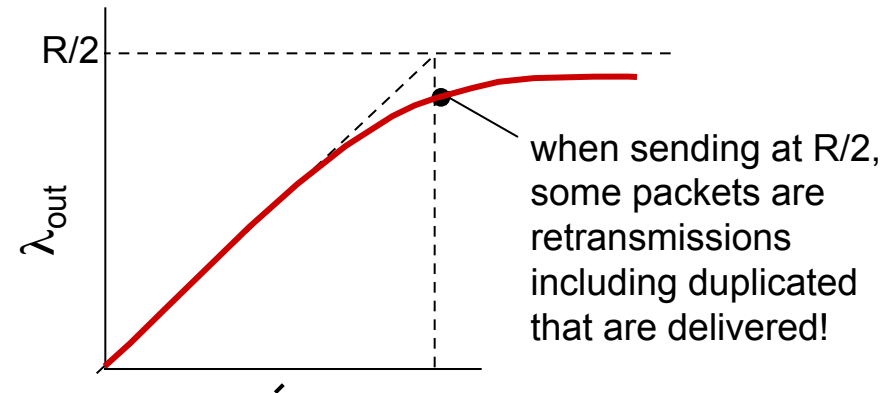


Causes/costs of congestion:

scenario 2

Realistic: duplicates

- ❖ packets can be lost, dropped at router due to full buffers
- ❖ sender times out



which

"costs" of congestion:

- ❖ more work (retrans) for given "goodput"
- ❖ unneeded retransmissions: link carries multiple copies of pkt
 - decreasing goodput

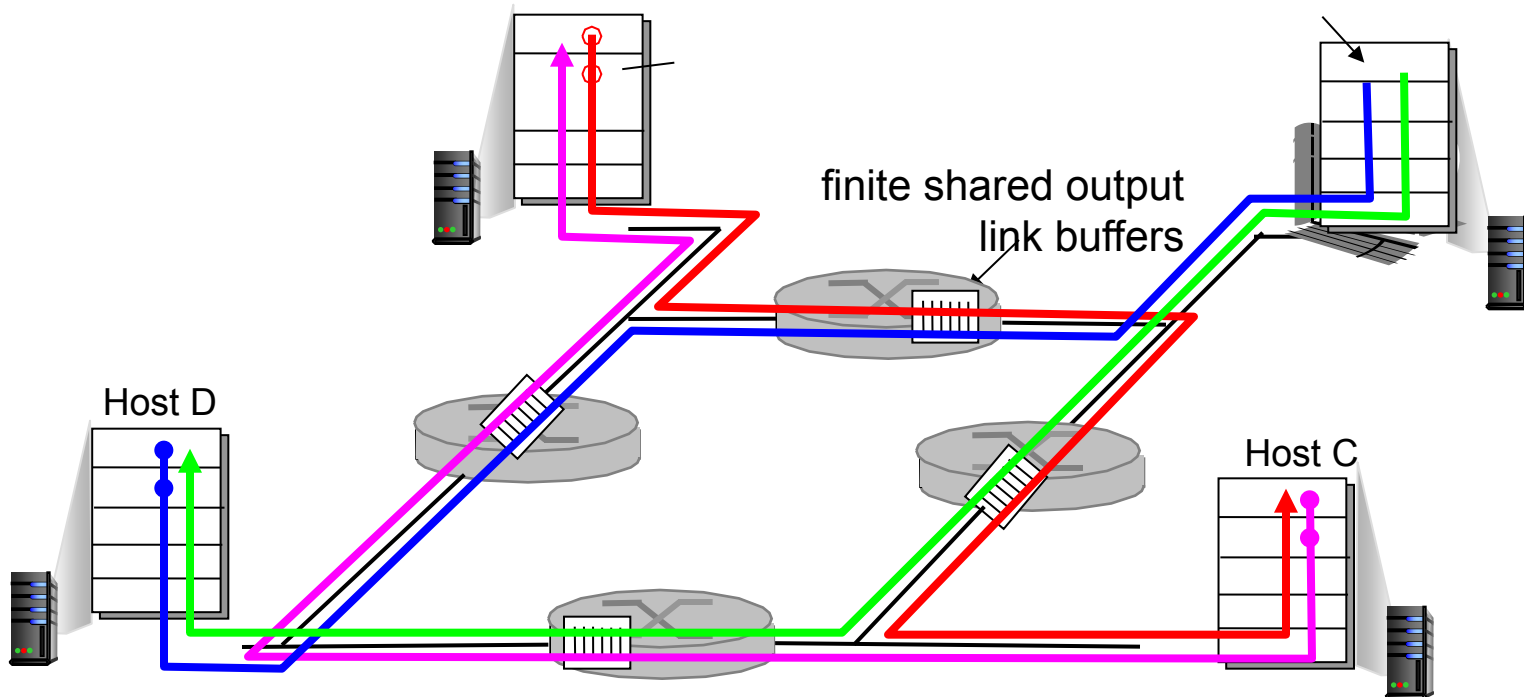
Causes/costs of congestion:

scenario 3

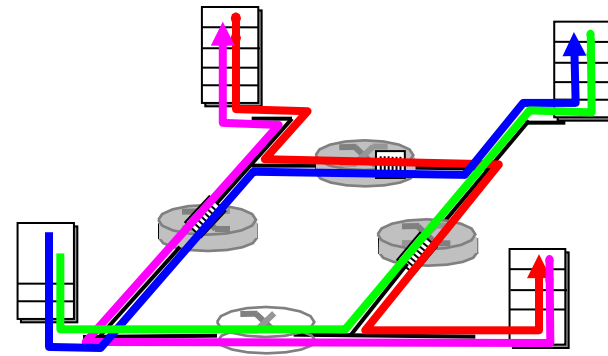
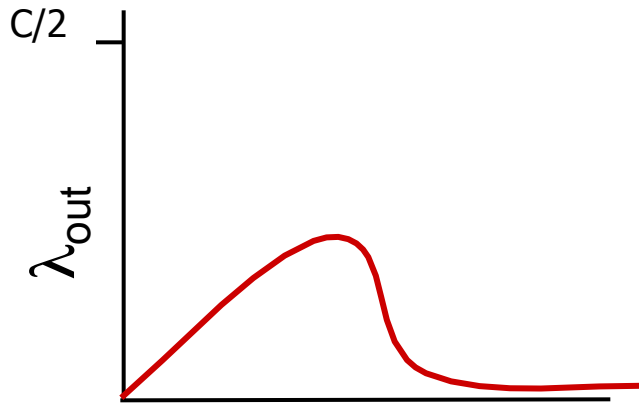
- ❖ four senders
- ❖ multihop paths
- ❖ timeout/retransmit

Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper re dropped, blue



Causes/costs of congestion: ~~scenario 3~~



another “cost” of congestion:

- ❖ when packet dropped, any “upstream transmission capacity used for that packet was wasted!

Approaches towards congestion control

two broad approaches towards congestion control:

end-end

- ❖ no end-to-end feedback from network
- ❖ congestion inferred from end-system observed loss, delay
- ❖ approach taken by TCP

network-assisted

- ❖ end-to-end feedback to end systems
 - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
 - explicit rate for sender to send at

Case study: ATM ABR congestion control

ABR: available bit rate:

- ❖ “elastic service

- sender can use available bandwidth

- ❖ if sender's path congested:

- sender throttled to minimum guaranteed rate

RM (resource management) cells:

from sender,

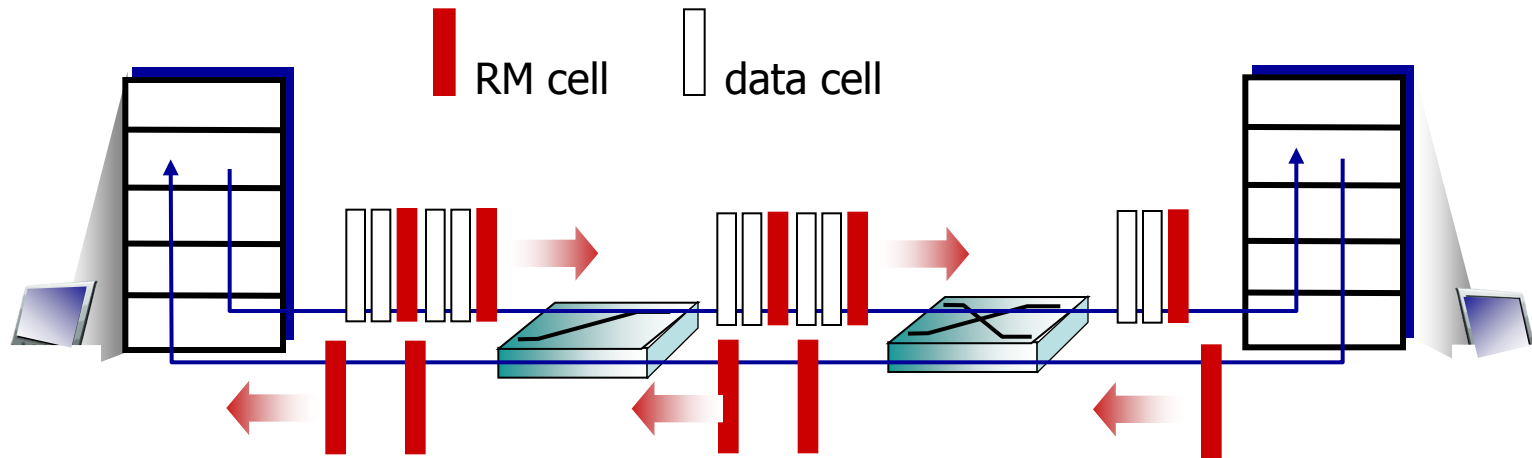
set by switches (“*network-assisted*”)

- *NI bit*: no increase in rate (mild congestion)

- *CI bit*: congestion indication

- ❖ RM cells returned to sender by receiver, with

Case study: ATM ABR congestion control



- ❖ two-byte ER (explicit rate) field in RM cell
 - congested switch may lower ER value in cell
 - senders' send rate thus max supportable rate on path
- ❖ EFCI bit in data cells: set to 1 in congested switch
 - if data cell preceding RM cell has EFCI set, receiver sets CI bit in returned RM cell

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexin

3.3 co
transport: UDP

3.4 principles of
reliable data
transfer

3.5 connection-
oriented transport:

- flow control
- connection management

3.6 principles of
congestion control

3.7 TCP congestion
control

TCP congestion control: additive increase multiplicative decrease

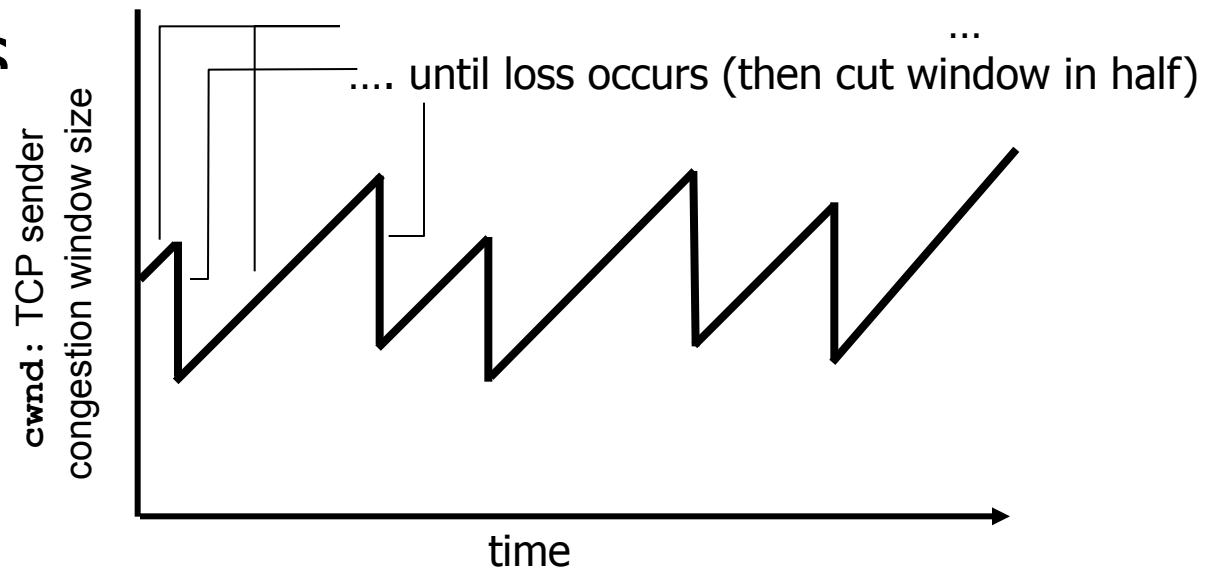
❖ *approach*: sender increases transmission rate (window size), probing for usable bandwidth, until loss occurs

■ *additive inc* $\quad \quad \quad \text{e cwnd by 1}$

■

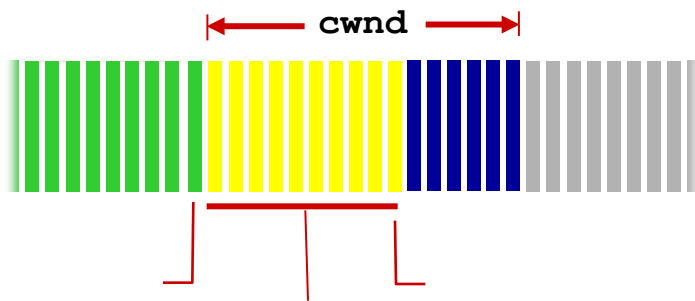
after loss

AIMD saw tooth behavior: probing for bandwidth



TCP Congestion Control: details

sender sequence number space



TCP sending rate:
❖ *roughly: send
wnd bytes, wait*

❖ sender limits

transmission:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

$$\text{rate} \approx \frac{\text{wnd}}{\text{RTT}} \text{ bytes/sec}$$

❖ **cwnd** is dynamic,
function of perceived
network congestion

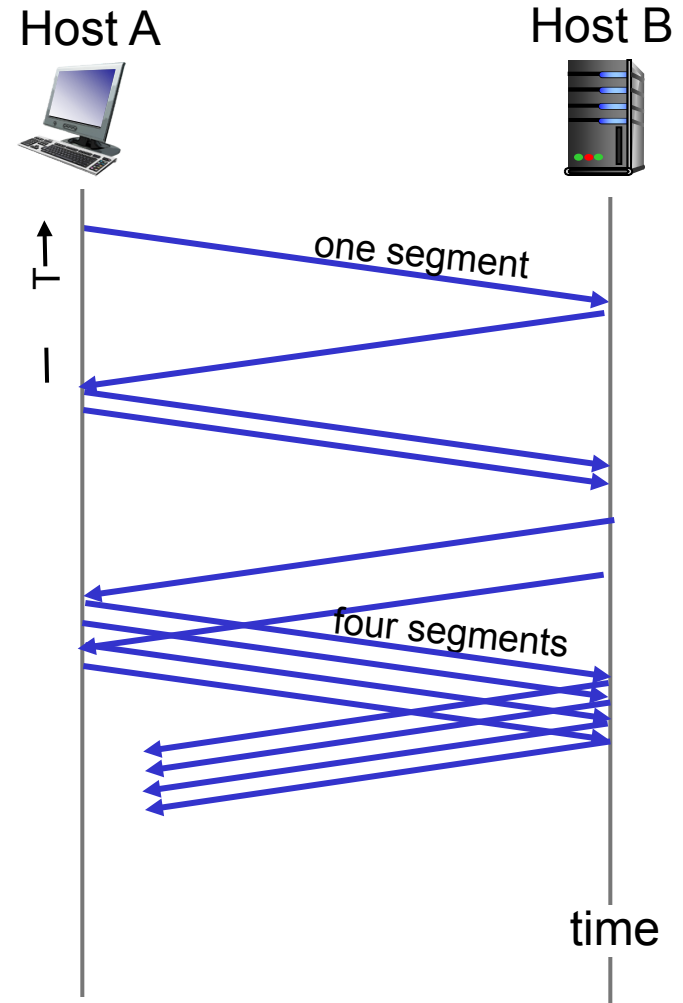
TCP Slow Start

- ❖ when connection begins, increase rate exponentially until first loss

RT

- done by incrementing `cwnd` for every ACK received

- ❖ summary: initial rate is slow but ramps up exponentially fast

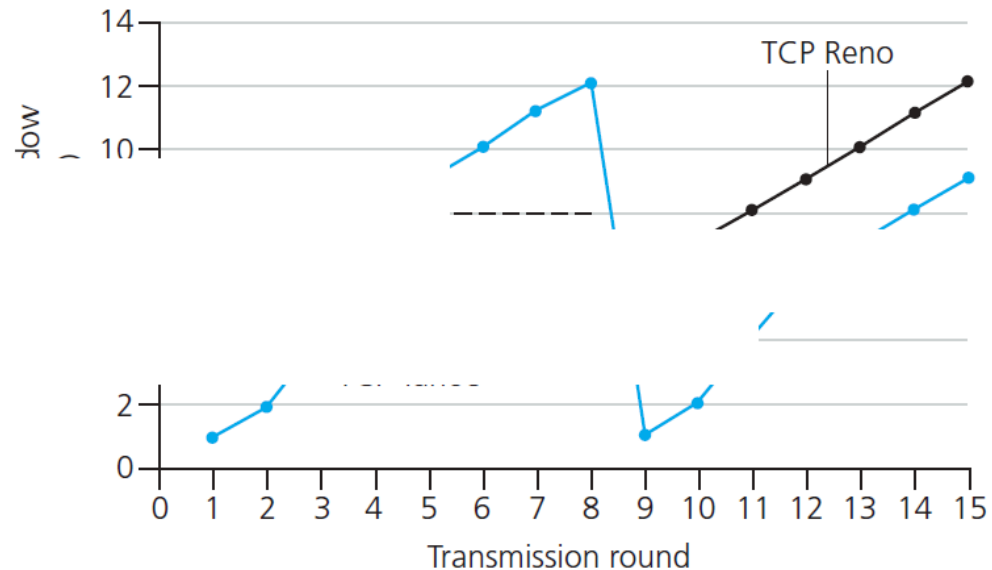


TCP: detecting, reacting to loss

- ❖ loss indicated by timeout:
 - `cwnd` set to 1 MSS;
 - window the entially (as in
- ❖ loss i Ks: TCP
RENO
 - dup ACKs indicate network capable of delivering some segments
 - `cwnd` is cut in half window then grows linearly
- ❖ TCP Tahoe always sets `cwnd` to 1 (timeout or 3 duplicate acks)

TCP: switching from slow start to CA

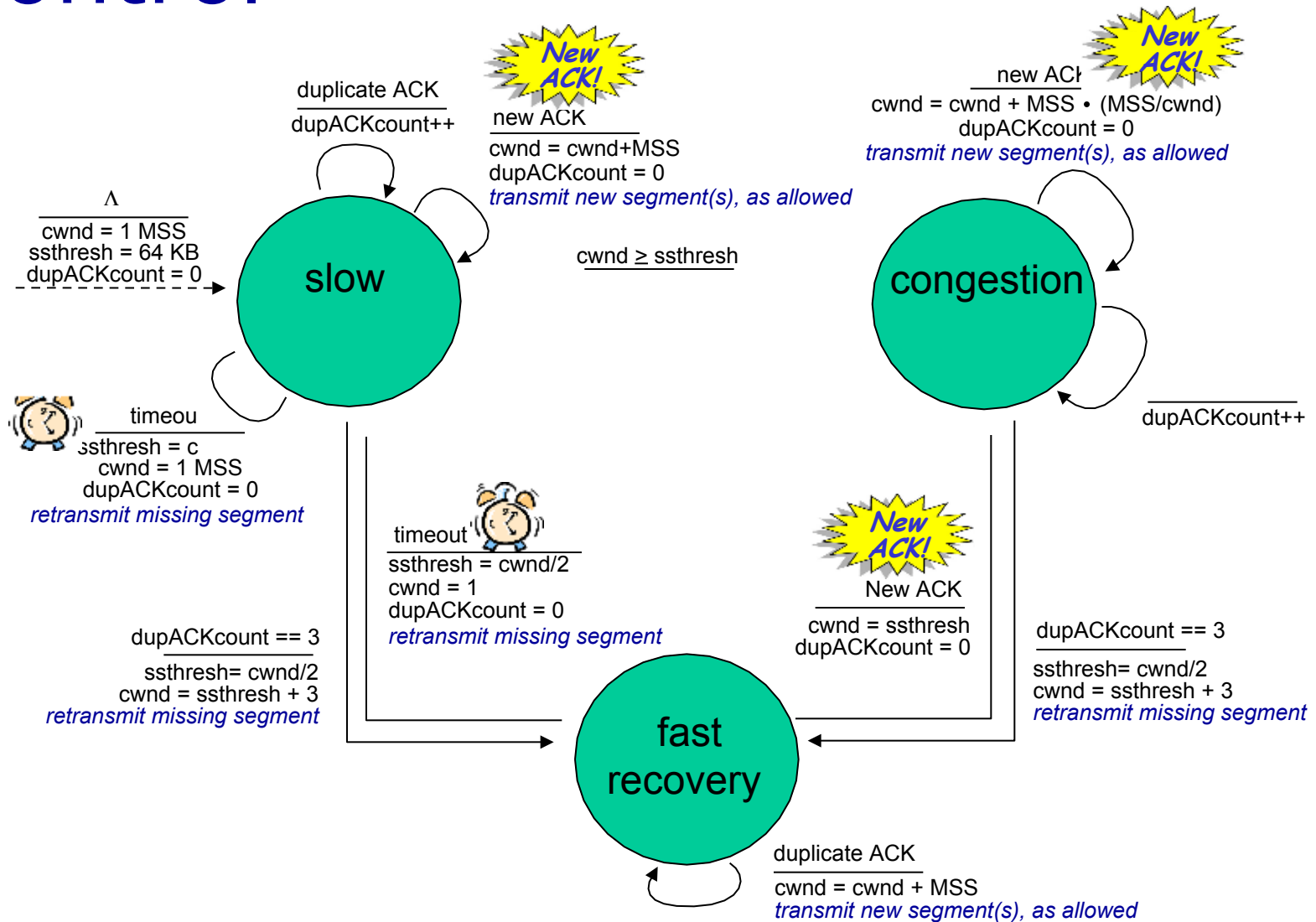
Q: when should the exponential increase switch to



gets to
its value
before
Implementation

- ❖ variable `ssthresh`
- ❖ on loss event, `ssthresh` is set to $1/2$ of `cwnd` just before loss event

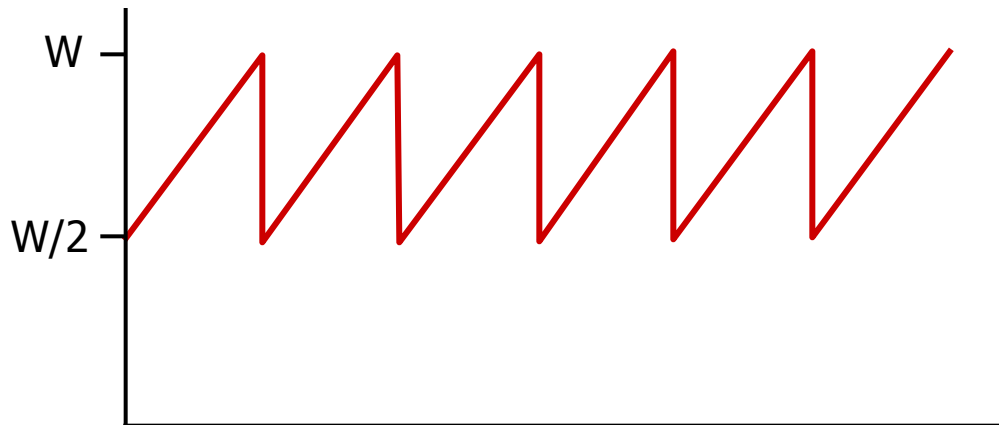
Summary: TCP Congestion Control



TCP throughput

- ❖ avg. TCP throughput as function of window size, RTT?
 - ignore slow start, assume always data to send
- ❖ W: window size, s = srtt where loss

- av $\frac{3}{4} W$
- avg. thruput is $\frac{3}{4}W$ per RTT



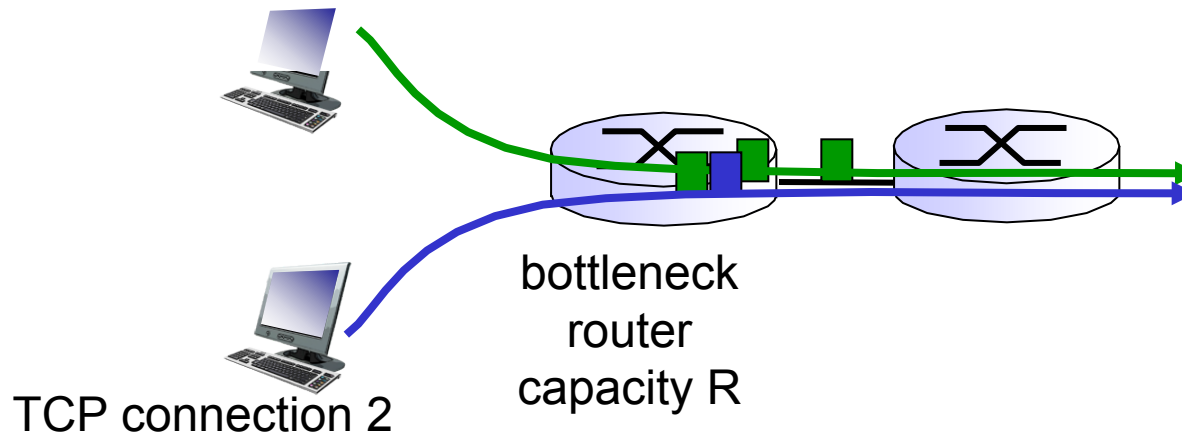
TCP Futures: TCP over “long, fat pipes”

- ❖ example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput
- ❖ requires $W =$ _____ ht segments

$$\text{TCP throughput} = \frac{L}{\text{RTT} \sqrt{L}}$$

TCP Fairness

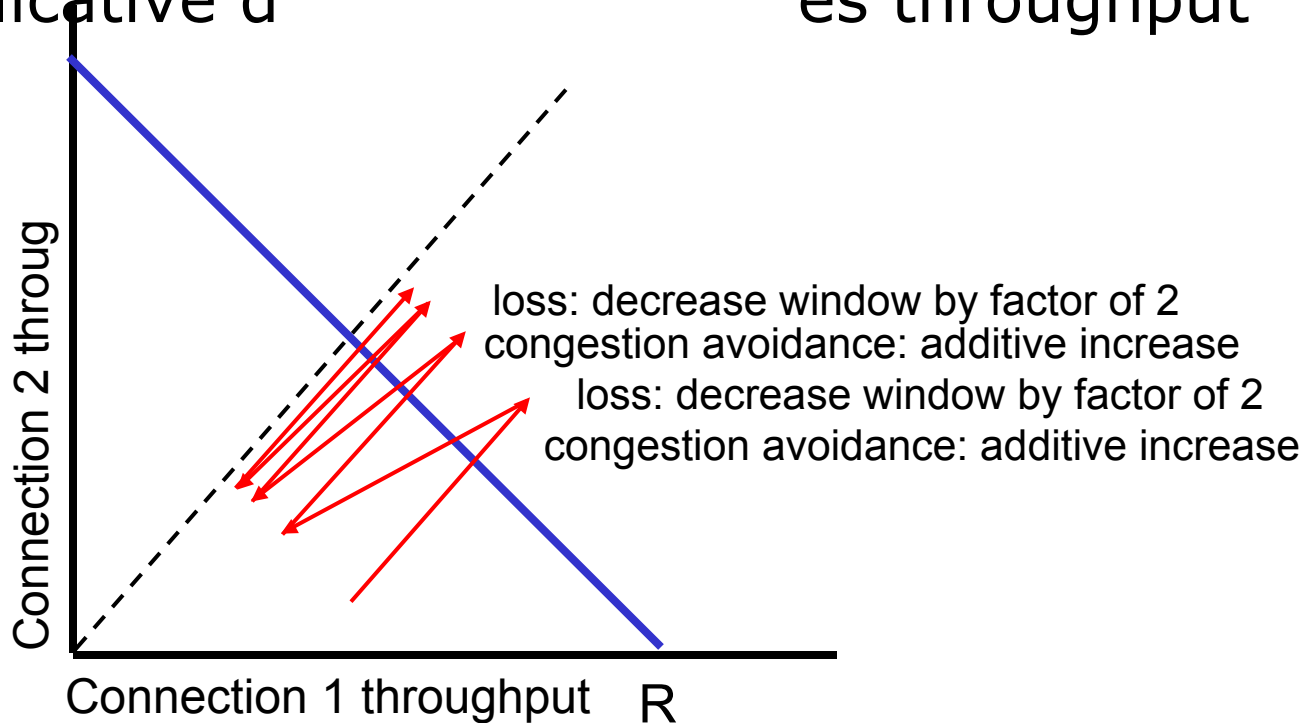
fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



Why is TCP fair?

two competing sessions:

- ❖ additive increase gives slope of 1, as throughput increases
- ❖ multiplicative decrease



Fairness (more)

Fairness and UDP

- ❖ multimedia apps often do not use TCP

con

- ❖ instead use UDP:
 - send audio/video at constant rate, tolerate packet loss

Fairness, parallel TCP connections

- ❖ application can open multiple parallel

- ❖ web browsers do this
- ❖ e.g., link of rate R with 9 existing connections:
 - new app asks for 1 TCP, gets rate $R/10$
 - new app asks for 11 TCPs, gets $R/2$

Chapter 3: summary

- ❖ principles behind transport layer services:

- multiplexin

- rel
transfer

- flow control

- congestion control

- ❖ instantiation, implementation in the Internet

- UDP

- TCP

ext:

“

tion,
transport
layers)

- ❖ into the network “core”